

Efficient Polynomial Evaluation over Structured Space and Application to Polynomial Method

Fukang Liu¹, Vaibhav Dixit², Daisuke Yamamoto¹, Wakaha Ogata¹,
Santanu Sarkar², Willi Meier³

¹ Institute of Science Tokyo, Tokyo, Japan

{liu.f.ad,ogata.w.aa}@m.titech.ac.jp,dai2000529yama@au.com

² IIT Madras, India

vaibhavrdk@gmail.com, sarkar.santanu.bir1@gmail.com

³ FHNW, Windisch, Switzerland

willimeier48@gmail.com

Abstract. It is well-known that evaluating a Boolean polynomial f of any degree d in n variables over the full space $\mathbb{F}_2^n$ takes $n \cdot 2^n$ bit operations and 2^n bits of memory with standard Möbius transform. When d is relatively small, Bouillaguet et al. proposed at CHES 2010 the fast exhaustive search (FES) algorithm. In this algorithm, by using Gray code to enumerate all elements in $\mathbb{F}_2^n$, evaluating f on all inputs in $\mathbb{F}_2^n$ takes $(\sum_{i=0}^d \binom{n}{i})^2 + d \cdot 2^n = \binom{n}{\leq d}^2 + d \cdot 2^n$ bit operations and $\binom{n}{\leq d}$ bits of memory. The term $\binom{n}{\leq d}^2$ represents the cost of the initialization phase. This problem has received new attention in recent years, which was studied by Dinur at EUROCRYPT 2021, by Furue and Takagi at PQCrypto 2023, and by Bouillaguet at TOMS 2024. All these algorithms work on the full space, and have a similar additional phase such as the initialization phase in the FES algorithm, which takes much more than $\binom{n}{\leq d}$ bit operations. In this work, we propose a simple yet efficient algorithm to evaluate f over the structured space $P_{n_s}^{w_s} \times \cdots \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n$ where $\sum_{i=1}^s n_i = n$ and $P_{n_i}^{w_i}$ denotes the set of n_i -bit binary strings with Hamming weight not larger than w_i . Our algorithm is inspired by the FES algorithm and Furue-Takagi's algorithm. However, our algorithm can work on a more general space, and is also distinguished by an efficient additional phase, which is simply reading all coefficients of f and thus takes only $\binom{n}{\leq d}$ bit operations. For complexity, our algorithm takes $\binom{n}{\leq d} + d \cdot \prod_{i=1}^s \binom{n_i}{\leq w_i}$ bit operations and consumes $2 \cdot \binom{n}{\leq d}$ bits of memory. For applications, we prove that it is either infeasible or nontrivial to adapt the FES algorithm with monotone Gray code, which somehow answers a question raised by Dinur at EUROCRYPT 2021. Moreover, our algorithm provides a proven method to solve a critical step in Dinur's algorithm for the polynomial method, without affecting its time complexity. In particular, we also address the open problem proposed at TOMS 2024, and improve the polynomial evaluation algorithms even over the full space.

Keywords: Boolean function, polynomial evaluation, Möbius transform, polynomial method

1 Introduction

We study the problem to efficiently evaluate a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d over a given input set S . It is well-known that when $S = \mathbb{F}_2^n$, standard Möbius transform can solve this problem with time complexity of $n \cdot 2^n$ bit operations and memory complexity of 2^n bits. A slightly modified variant by Dinur and Shamir can achieve time complexity of $d \cdot 2^n$ bit operations and memory complexity of still 2^n bits [6].

To address the issue of the huge memory consumption in standard Möbius transform, 5 different memory-efficient algorithms have been developed in the literature, and they can be classified into two types. The first type was first proposed by Bouillaguet et al. at CHES 2010 [3], which is called the fast exhaustive search (FES) algorithm. It uses Gray code to enumerate all inputs in $\mathbb{F}_2^n$ and exploits the derivatives of f to achieve efficient polynomial evaluations. We call this type the derivative-based polynomial evaluations. This FES algorithm takes $\binom{n}{\leq d}^2 + d \cdot 2^n$ bit operations and $\binom{n}{\leq d}$ bits of memory. The term $\binom{n}{\leq d}^2$ is the cost of the initialization phase of the algorithm. Building on the FES framework, Furue and Takagi proposed at PQCrypt 2023 a different order to enumerate all inputs in $\mathbb{F}_2^n$, and also achieved time complexity of $\binom{n}{\leq d}^2 + d \cdot 2^n$ bit operations [7], though the memory complexity is increased to $\log n \cdot n \cdot \binom{n}{\leq d}$ bits. Moreover, Furue-Takagi's technique also works on a more general vector space $\mathbb{F}_p^n$ where p is a prime number. Both derivative-based polynomial evaluation algorithms [3, 7] have a costly initialization phase, i.e., in order to compute $\binom{n}{\leq d} - 1$ different derivatives of f with time complexity upper bounded by $\binom{n}{\leq d}^2$.

For large d , both derivative-based polynomial evaluation algorithms become inefficient due to the costly initialization phase. This has motivated Dinur to develop memory-efficient Möbius transform at EUROCRYPT 2021 [4] where he proposed his algorithm for the polynomial method [10] (we call Dinur's polynomial method for simplicity). Specifically, the last step in Dinur's polynomial method is to perform polynomial evaluations for a polynomial of a relatively high degree $d \approx n/3$ over $\mathbb{F}_2^n$. The idea of the memory-efficient Möbius transform is to fix some variables in f first, and then update the polynomial accordingly to obtain an updated polynomial in fewer variables, and finally perform standard Möbius transform on this updated polynomial. The cost to update the polynomial for all possible values of the to-be-fixed variables is non-negligible, resulting in additional cost as in derivative-based polynomial evaluations (i.e., the initialization phase), though the additional cost for updating the polynomial can always be kept below $n \cdot 2^n$. Bouillaguet further explored this fix-and-update method at [2], and achieved the time complexity of $2d \cdot 2^n$ bit operations and memory complexity of $\binom{n}{\leq d}$ bits, where the additional cost $d \cdot 2^n$ is consumed by updating the polynomial.

Motivation. First, there is a critical step in Dinur’s polynomial method [4], which is to perform polynomial evaluations over the set

$$\{(x_n, \dots, x_{n-n_1+1}) \in \mathbb{F}_2^{n-n_1+1} \mid \text{Hw}(x_n, \dots, x_{n-n_1+1}) \leq w_2\} \times \{0, 1\}^{n_1}$$

rather than all inputs in $\mathbb{F}_2^n$, where $\text{Hw}(x_n, \dots, x_1)$ is the Hamming weight of $(x_n, \dots, x_1) \in \mathbb{F}_2^n$. Since monotone Gray code [11] can traverse the binary vectors (almost) in increasing Hamming weight order, Dinur asked whether it is possible to adapt the FES algorithm [3] with monotone Gray code. Although monotone Gray code was not used to solve this critical step, Dinur claimed that a slightly tweaked version of the FES algorithm [3] with Gray code can solve this critical step with time complexity $d \cdot \binom{n-n_1}{\leq w_2} \cdot 2^{n_1}$ as long as n is much smaller than 2^{n_1} , i.e., $n \ll 2^{n_1}$ holds in Dinur’s polynomial method. The details can be found on Page 9 of [4]. It is also claimed that it is nontrivial to achieve this time complexity for any n_1 . On the other hand, the correctness of this slightly modified FES algorithm was not rigorously proved, and details of how this algorithm actually runs were not given. We thus started this research in order to address these issues. This is important since Dinur’s polynomial method to solve Boolean equations has a significant advantage over known solving techniques and has been applied to cryptanalysis of several popular ciphers including SHA-3 [4], LowMC [1, 4, 9] and Rain [8].

Second, in the latest research on polynomial evaluations [2], Bouillaguet stated an open problem to develop a memory-efficient algorithm⁴ for polynomial evaluations over the set $\{x \in \mathbb{F}_2^n \mid \text{Hw}(x) \leq w\}$ since it has an important application to a critical step of Dinur’s polynomial method stated above. We are thus motivated to dive into the FES algorithm framework [3] and explore whether new insight can be obtained to address this question.

Lastly, existing memory-efficient polynomial evaluation algorithms [2–4, 7] require additional costly phases, i.e., either the initialization phase or the polynomial update, which is also not friendly to implementation. Can we develop such an algorithm that it is as simple as standard Möbius transform, where after reading the coefficients of f , the remaining step is simply updating some entries of arrays for each input to f ? This motivates us to develop a simple yet efficient algorithm.

Our contribution. We dive into the FES framework proposed by Bouillaguet et al. in [3], and rephrase it into a simpler (non-recursive) form. Based on new concepts called evaluation rule and evaluation order derived from this new form, we propose a sufficient condition for this framework.

Based on the new insight into the FES framework, we prove that it is either infeasible or nontrivial to adapt the FES algorithm [3] with monotone Gray code, which somehow answers a question raised by Dinur in [4]. Moreover, we can use a proven memory-efficient algorithm to replace the (unproved)

⁴The standard Möbius transform can solve this problem, but the memory complexity is $\binom{n}{\leq w}$. Thus, it is not memory-efficient.

tweaked FES algorithm [4] to solve a critical step in Dinur's polynomial method stated above, without affecting its time complexity. In particular, there is not any assumption on n_1 in order to obtain the same time complexity.

Specifically, we consider a more general problem of how to efficiently evaluate a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d over the space $P_{n_s}^{w_s} \times \dots \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n$ where $P_{n_i}^{w_i} = \{x \in \mathbb{F}_2^{n_i} \mid \text{Hw}(x) \leq w_i\}$. Based on new insight into the FES algorithm, we successfully find an efficient algorithm for this problem, where the initialization phase is simply reading coefficients of f , just as in standard Möbius transform. Therefore, we address the issue of the costly initialization phases in [3, 7]. The time complexity and memory complexity of our algorithm are $\binom{n}{\leq d} + d \cdot \prod_{i=1}^s \binom{n_i}{\leq w_i}$ bit operations and $2 \cdot \binom{n}{\leq d}$ bits, respectively.

For applications, it directly solves the critical step in Dinur's polynomial method [4] by setting $s = 2, w_1 = n_1$. Moreover, we answer the open problem raised by Bouillaguet in [2] by setting $s = 1, n_1 = n, w_1 \leq n$. In addition, as shown in Table 1, we also achieve the fastest algorithm for polynomial evaluations on all inputs in $\mathbb{F}_2^n$ by setting $s = 1, n_1 = n, w_1 = n$.

Organization. In Section 2, we introduce the notation, the Gray code, monotone Gray code and existing polynomial evaluation algorithms. In Section 3, we present our new algorithm, analyze its complexity and demonstrate its applications. In Section 4, we rephrase the FES framework, and explain the technical issue of using monotone Gray code in this framework. In Section 5, we prove the correctness of our algorithm. Finally we conclude the paper in Section 6.

2 Preliminaries

2.1 Notation

The following notation will be used throughout this paper.

1. The size of a set S is denoted by $|S|$, and we also view a set as an array, where $S = \{S[0], \dots, S[|S| - 1]\}$, i.e., $S[i]$ is the $(i + 1)$ -th element in S .
2. $\oplus, \wedge$ and $\gg$ denote the exclusive OR, AND and logical right shift, respectively.
3. $\mathbf{0}^n$ represents the all-zero vector $(0, \dots, 0) \in \mathbb{F}_2^n$.
4. The sum $\sum_{i=0}^d \binom{n}{i}$ is denoted by $\binom{n}{\leq d}$.
5. The set of integers $\{i, i + 1, \dots, j\}$ is denoted by $[i, j]$.
6. The bold character $\mathbf{x}$ is used to represent a Boolean vector. In this paper, we set $\mathbf{x} = (x_n, \dots, x_1) \in \mathbb{F}_2^n$.
7. $\mathbf{e}_i = (0, \dots, 0, \underbrace{1, 0, \dots, 0}_{i-1}) \in \mathbb{F}_2^n$ and $\bar{\mathbf{e}}_i = (1, \dots, 1, 0, \underbrace{1, \dots, 1}_{i-1}) \in \mathbb{F}_2^n$ where $1 \leq i \leq n$.
8. The Hamming weight of the Boolean vector $\mathbf{x}$ is denoted by $\text{Hw}(\mathbf{x})$ and it is defined as the number of nonzero elements in $\mathbf{x}$, i.e.,

$$\text{Hw}(\mathbf{x}) = \#\{i \mid x_i = 1 \text{ for } i \in [1, n]\}.$$

9. The lex order between two Boolean vectors $\mathbf{x}$ and $\mathbf{y}$ is defined as follows:

$$\mathbf{x} <_{\text{lex}} \mathbf{y} \Leftrightarrow \exists j \in [1, n] : x_i = y_i \text{ for } i \in [j+1, n], y_j = 1, x_j = 0.$$

10. We define the function $INT(\mathbf{x})$ as $INT(\mathbf{x}) = \sum_{i=1}^n 2^{i-1} x_i$, i.e., $\mathbf{x}$ is the n -bit binary representation of the integer $INT(\mathbf{x})$.

In this paper, we focus on Boolean polynomials. As is well-known, for a Boolean polynomial $f(x_n, \dots, x_1)$, its algebraic normal form (ANF) can be expressed as

$$f(x_n, \dots, x_1) = \sum_{(b_n, \dots, b_1) \in \mathbb{F}_2^n} \mu(\mathbf{b}) \prod_{i=1}^n x_i^{b_i} \quad (1)$$

where $\mathbf{b} = (b_n, \dots, b_1) \in \mathbb{F}_2^n$ and $\mu(\mathbf{b}) \in \mathbb{F}_2$ denotes the coefficient of the monomial $x_1^{b_1} x_2^{b_2} \dots x_n^{b_n}$. If the polynomial is of degree d , we have $\mu(\mathbf{b}) = 0$ for $\text{Hw}(\mathbf{b}) > d$. Throughout this paper, the polynomial $f(x_n, \dots, x_1)$ will be written as either f or $f(\mathbf{x})$, which should be clear from the contexts.

2.2 Gray Code and Monotone Gray Code

We first recall Gray code and monotone Gray code⁵.

Definition 1 (Gray Code). The Gray code (also called reflected binary Gray code) for n -bit binary vectors denoted by G_n is an ordered set of all $\mathbf{x} \in \mathbb{F}_2^n$ where

$$\forall \mathbf{x} \in \mathbb{F}_2^n : G_n[INT(\mathbf{x})] = \mathbf{x} \oplus (\mathbf{x} >> 1).$$

In particular, G_n has the following property:

$$\forall j \in [1, 2^n - 1] : \text{Hw}(G_n[j-1] \oplus G_n[j]) = 1.$$

Monotone Gray code was proposed in [11], which was explained from the perspective of a graph. Here, we are only interested in its construction, and give an equivalent description from the perspective of sets of elements in $\mathbb{F}_2^n$.

Definition 2 (Construction of Monotone Gray Code [11]). Let $V_{0,1}$ be an ordered set of size $s_{0,1} = 2$ and

$$V_{0,1} = \{\mathbf{x}, \mathbf{y}\} \text{ where } \text{Hw}(\mathbf{x} \oplus \mathbf{y}) = 1, 0 \leq \text{Hw}(\mathbf{x}), \text{Hw}(\mathbf{y}) \leq 1.$$

Let $V_{i,i+1}$ ($1 \leq i \leq n-1$) be an ordered set of size $s_{i,i+1} \geq 2$, where

$$\begin{cases} \text{Hw}(V_{i-1,i}[s_{i-1,i}-1] \oplus V_{i,i+1}[0]) = 1 \\ \text{Hw}(V_{i,i+1}[j] \oplus V_{i,i+1}[j+1]) = 1 \text{ for } j \in [0, s_{i,i+1}-2] \\ \text{Hw}(V_{i,i+1}[j]) \in [i, i+1] \text{ for } j \in [0, s_{i,i+1}-1] \\ V_{i-1,i} \cap V_{i,i+1} = \emptyset, \\ \{\mathbf{x} \in \mathbb{F}_2^n \mid \text{Hw}(\mathbf{x}) = i\} \subseteq V_{i-1,i} \cup V_{i,i+1}. \end{cases} \quad (2)$$

⁵A related lecture is also available at https://ktiml.mff.cuni.cz/~gregor/hypercube/hypercubes_lec10.pdf.

Then, the ordered set MG_n is the monotone Gray code for all $x \in \mathbb{F}_2^n$, where

$$MG_n = \{ V_{0,1}[0], \dots, V_{0,1}[s_{0,1} - 1], \dots, V_{i-1,i}[0], \dots, V_{i-1,i}[s_{i-1,i} - 1], \\ V_{i,i+1}[0], \dots, V_{i,i+1}[s_{i,i+1} - 1], \dots, V_{n-1,n}[0], \dots, V_{n-1,n}[s_{n-1,n} - 1] \}.$$

It is easy to observe that the ordered set

$$MG_{n,i} = \{ V_{0,1}[0], \dots, V_{0,1}[s_{0,1} - 1], \dots, V_{i,i+1}[0], \dots, V_{i,i+1}[s_{i,i+1} - 1] \}$$

has the property $\{x \in \mathbb{F}_2^n \mid \text{Hw}(x) \leq i\} \subseteq MG_{n,i}$.

Based on the construction of monotone Gray code, we propose a useful lemma that can later be used to prove that it is either infeasible or nontrivial to adapt the FES algorithm [3] with monotone Gray code, which is stated as an interesting question by Dinur in [4].

Lemma 1. When $|V_{i,i+1}| = s_{i,i+1} \geq n + 2$, for any $0 \leq j \leq s_{i,i+1} - n - 2$, among the $(n + 1)$ ordered pairs of two consecutive elements in $n + 2$ consecutive elements $V_{i,i+1}[j], \dots, V_{i,i+1}[j + n + 1]$ in $V_{i,i+1}$, there are at least two ordered pairs $(V_{i,i+1}[j_1], V_{i,i+1}[j_1 + 1])$ and $(V_{i,i+1}[j_2], V_{i,i+1}[j_2 + 1])$ satisfying

$$j_1, j_2 \in [j, j + n], j_1 \neq j_2, V_{i,i+1}[j_1] \oplus V_{i,i+1}[j_1 + 1] = V_{i,i+1}[j_2] \oplus V_{i,i+1}[j_2 + 1].$$

In particular, let $V_{i,i+1}[j_1] \oplus V_{i,i+1}[j_1 + 1] = e_k$ for each such (j_1, j_2) . Then, we have

$$V_{i,i+1}[j_1] \wedge \overline{e_k} = V_{i,i+1}[j_1 + 1] \wedge \overline{e_k}, V_{i,i+1}[j_2] \wedge \overline{e_k} = V_{i,i+1}[j_2 + 1] \wedge \overline{e_k},$$

and $\text{Hw}(V_{i,i+1}[j_1 + 1] \wedge \overline{e_k}) = \text{Hw}(V_{i,i+1}[j_2 + 1] \wedge \overline{e_k}) = i$.

Proof. Based on the constraints in Equation 2, we have that

$$\forall j \in [0, s_{i,i+1} - 2] : \text{Hw}(V_{i,i+1}[j] \oplus V_{i,i+1}[j + 1]) = 1.$$

If $|V_{i,i+1}| = s_{i,i+1} \geq n + 2$, for any $0 \leq j \leq s_{i,i+1} - n - 2$, there are $n + 2$ consecutive elements $V_{i,i+1}[j], \dots, V_{i,i+1}[j + n + 1]$ in $V_{i,i+1}$. Hence, there will be $n + 1$ integers $(k_j, \dots, k_{j+n})$ ($1 \leq k_j, \dots, k_{j+n} \leq n$) such that

$$\forall j_3 \in [j, j + n] : V_{i,i+1}[j_3] \oplus V_{i,i+1}[j_3 + 1] = e_{k_{j_3}}.$$

The $(n + 1)$ ordered pairs of 2 consecutive elements are $(V_{i,i+1}[j], V_{i,i+1}[j + 1]), \dots, (V_{i,i+1}[j + n], V_{i,i+1}[j + n + 1])$. Due to $1 \leq k_j, \dots, k_{j+n} \leq n$ and $e_{k_{j_3}} \in \{e_1, \dots, e_n\}$, there must exist $j_1, j_2 \in [j, j + n]$ and $j_1 \neq j_2$ such that $k_{j_1} = k_{j_2}$. In other words,

$$\exists j_1, j_2 \in [j, j + n], j_1 \neq j_2 : V_{i,i+1}[j_1] \oplus V_{i,i+1}[j_1 + 1] = V_{i,i+1}[j_2] \oplus V_{i,i+1}[j_2 + 1].$$

For such (j_1, j_2) , let $V_{i,i+1}[j_1] \oplus V_{i,i+1}[j_1 + 1] = e_k$. Then, we have

$$V_{i,i+1}[j_1] \oplus V_{i,i+1}[j_1 + 1] = V_{i,i+1}[j_2] \oplus V_{i,i+1}[j_2 + 1] = e_k.$$

In other words, both pairs $(V_{i,i+1}[j_1], V_{i,i+1}[j_1 + 1])$ and $(V_{i,i+1}[j_2], V_{i,i+1}[j_2 + 1])$ differ in the k -th bit. Therefore, by setting the k -bit bit of each pair to 0, they become identical, resulting in

$$V_{i,i+1}[j_1] \wedge \overline{e_k} = V_{i,i+1}[j_1 + 1] \wedge \overline{e_k}, V_{i,i+1}[j_2] \wedge \overline{e_k} = V_{i,i+1}[j_2 + 1] \wedge \overline{e_k}.$$

Moreover, due to $\text{Hw}(V_{i,i+1}[j_1]), \text{Hw}(V_{i,i+1}[j_1 + 1]), \text{Hw}(V_{i,i+1}[j_2]), \text{Hw}(V_{i,i+1}[j_2 + 1]) \in \{i, i + 1\}$, we have $\text{Hw}(V_{i,i+1}[j_1 + 1] \wedge \overline{e_k}) = \text{Hw}(V_{i,i+1}[j_2 + 1] \wedge \overline{e_k}) = i$. $\square$

2.3 Algorithms for Polynomial Evaluations

In this section, we focus on a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d .

Standard Möbius transform. The standard Möbius transform can be used to evaluate f on all inputs $x \in \mathbb{F}_2^n$ with 2^n bits of memory and $n \cdot 2^n$ bit operations. There are initialization phase and evaluation phase. At the initialization phase, an array of size 2^n denoted by B is constructed and initialized as follows:

$$\forall (x_n, \dots, x_1) \in \mathbb{F}_2^n : B[\text{INT}(x_n, \dots, x_1)] = \mu(x_n, \dots, x_1).$$

Then, move to the evaluation phase that consists of the following n steps:

- Step 1: For each $(x_n, \dots, x_1) \in \mathbb{F}_2^n$ with $x_1 = 1$, update B as follows:

$$B[\text{INT}(x_n, \dots, x_2, 1)] = B[\text{INT}(x_n, \dots, x_2, 0)] \oplus B[\text{INT}(x_n, \dots, x_2, 1)].$$

- Step $i \in [2, n]$: For each $(x_n, \dots, x_1) \in \mathbb{F}_2^n$ with $x_i = 1$, update B as follows:

$$\begin{aligned} & B[\text{INT}(x_n, \dots, x_1)] \\ &= B[\text{INT}(x_n, \dots, x_{i+1}, 0, x_{i-1}, \dots, x_1)] \oplus B[\text{INT}(x_n, \dots, x_1)]. \end{aligned}$$

After the evaluation phase, there will be $B[\text{INT}(x_n, \dots, x_1)] = f(x_n, \dots, x_1)$ for $\forall (x_n, \dots, x_1) \in \mathbb{F}_2^n$.

If the polynomial f is of degree d , Dinur and Shamir have shown in [6] that the time complexity of the standard Möbius transform can be reduced to $d \cdot 2^n$, but the memory complexity is still 2^n .

Other memory-efficient algorithms. To address the issue of massive memory consumption in standard Möbius transform, memory-efficient algorithms to evaluate polynomials on all inputs over $\mathbb{F}_2^n$ have been proposed.

1. Fast exhaustive search [3]: This algorithm was proposed by Bouillaguet et al. at CHES 2010, which is based on Gray code to traverse $\mathbb{F}_2^n$, and exploits the derivative of f up to the d -th order derivative. The time complexity is $\binom{n}{\leq d}^2 + d \cdot 2^n$ and the memory complexity is $\binom{n}{\leq d}$.

2. Fast enumeration algorithm [7]: This algorithm was proposed by Furue and Takagi to evaluate a polynomial on all inputs in $\mathbb{F}_p^n$ where p is a prime number. It is based on the lex order to traverse $\mathbb{F}_p^n$. Similar to FES, this algorithm also exploits the derivative of f up to the d -th order derivative. For $p = 2$, the time complexity and memory complexity are $\binom{n}{\leq d}^2 + d \cdot 2^n$ and $\log(n) \cdot n \cdot \binom{n}{\leq d}$, respectively.
3. Memory-efficient Möbius transform [4]: This algorithm was proposed by Dinur at EUROCRYPT 2021. It is based on a fix-and-update method. Specifically, update the original polynomial by fixing $n - d$ variables with all possible values, and then perform the standard Möbius transform for this updated polynomial in d variables. The memory complexity is less than $n \cdot \binom{n}{\leq d}$, and the time complexity is

$$\binom{n}{\leq d} + 2 \cdot \binom{n-1}{\leq d} + \dots + 2^{n-d-1} \cdot \binom{d+1}{\leq d} + 2^{n-d} \cdot d \cdot 2^d,$$

where the term $\binom{n}{\leq d} + 2 \cdot \binom{n-1}{\leq d} + \dots + 2^{n-d-1} \cdot \binom{d+1}{\leq d} > 2^{n-1}$ is the cost for the polynomial update. Obviously, the time complexity is smaller than $n \cdot 2^n$, but larger than $d \cdot 2^n + 2^{n-1}$.

4. Memory-efficient Möbius transform [4]: To further reduce the memory complexity, Dinur restricted that the number of variables to fix is $n' = n - \log \binom{n}{\leq d}$ and no intermediate polynomials are stored. The memory complexity becomes $2 \cdot \binom{n}{\leq d}$, and the time complexity becomes

$$2^{n'} \cdot \left(\binom{n}{\leq d} + \dots + \binom{n-n'}{\leq d} \right) + (n - n') \cdot 2^n < n \cdot 2^n.$$

5. Memory-efficient Möbius transform [2]. Bouillaguet further improved Dinur's memory-efficient Möbius transform by proposing an efficient algorithm to compute the updated polynomial in d variables after fixing $n - d$ variables. The memory complexity and memory complexity are $\binom{n}{\leq d}$ and $\in d \cdot 2^n$, respectively, since updating the polynomial takes about $d \cdot 2^n$ bit operations.

Summary. For the two derivative-based methods in [3,7], they require an initialization phase to compute $\binom{n}{\leq d}$ different derivatives of f . Although the authors of [3,7] are aware that $\binom{n}{\leq d}^2$ is a crude upper bound of the cost to compute these derivatives, they have not tried to address this issue (see also [2]). Moreover, as pointed out by Dinur in [4], this costly initialization phase is the main reason why he developed an alternate method to perform memory-efficient algorithms for polynomial evaluations.

As for memory-efficient Möbius transform in [2,4], they are based on the same strategy: fix some variables to obtain an updated polynomial in fewer variables, and then perform the standard Möbius transform for this updated polynomial in fewer variables. Therefore, they also require an additional phase,

i.e., fix variables and update the polynomial. Although the cost of this phase is less than $\binom{n}{\leq d}^2$ for large d and does not exceed $d \cdot 2^n$, it is still larger than $\binom{n}{\leq d}$.

For the standard Möbius transform, the initialization phase is simply reading coefficients of the polynomial. A natural question is whether it is possible to develop a memory-efficient algorithm whose initialization phase is also simply reading coefficients of the polynomial, and the evaluation phase is also simply updating some entries of the arrays, just as simple as Möbius transform.

3 New Memory-efficient Polynomial Evaluation

We consider the problem of evaluating a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d on all inputs in the following structured space denoted by $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$:

$$P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = P_{n_s}^{w_s} \times \dots \times P_{n_2}^{w_2} \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n, \quad n = \sum_{i=1}^s n_i,$$

$$\forall i \in [1, s] : P_{n_i}^{w_i} = \{\mathbf{x} \in \mathbb{F}_2^{n_i} \mid \text{Hw}(\mathbf{x}) \leq w_i\}.$$

Our algorithm is very simple, and thus we describe it first. The complexity analysis, the application of this algorithm, and the proof of its correctness will be discussed later. Our algorithm also consists of the initialization phase and the evaluation phase, as detailed below.

Initialization phase. Assign $\binom{n}{\leq d}$ different arrays, which are classified into $d + 1$ different types:

- Type-0: The array is denoted by A_0 . The size of A_0 is $n + 1$.
- Type- i ($1 \leq i \leq d - 1$): The array is denoted by $A_{0, k_1, \dots, k_i}$ where $1 \leq k_1 < \dots < k_i \leq n$. The size of $A_{0, k_1, \dots, k_i}$ is $n + 1 - k_i$.
- Type- d : The array is denoted by $A_{0, k_1, \dots, k_d}$ where $1 \leq k_1 < \dots < k_d \leq n$. The size of $A_{0, k_1, \dots, k_d}$ is 1.

Only initialize the first entry⁶ of each of these arrays as follows, which is equivalent to reading the coefficients $\mu(b_n, \dots, b_1)$ of f :

$$A_0[0] = \mu(\mathbf{0}^n),$$

$$\forall i \in [1, d] \text{ and } 1 \leq k_1 < \dots < k_i \leq n : A_{0, k_1, \dots, k_i}[0] = \mu\left(\sum_{j=1}^i \mathbf{e}_{k_j}\right).$$

Note that $\mu(\mathbf{0}^n)$ is the constant part in f , and $\mu(\sum_{j=1}^i \mathbf{e}_{k_j})$ is the coefficient of the monomial $x_{k_1} x_{k_2} \dots x_{k_i}$ in f where $1 \leq i \leq d$.

For convenience, throughout this paper, we set $k_0 = 0$. Hence, each of these arrays can be denoted by $A_{k_0, \dots, k_i}$ where $0 \leq k_0 < \dots < k_i \leq n$ and $0 \leq i \leq d$.

Before describing the evaluation phase, we introduce the following concept.

⁶In implementation, an array of size $\binom{n}{\leq d}$ can also be used to store all $A_{0, k_1, \dots, k_i}[0]$, reducing the size of each $A_{0, k_1, \dots, k_i}$ by 1. The total memory consumption is still the same.

Algorithm 1 Evaluation of $f(x)$ with $h = \text{Hw}(x)$ and $(k_h, \dots, k_1) = \text{Sup}(x)$

```

1: procedure EVA( $h, d, k_h, \dots, k_1$ )
2:   for  $i$  from  $d - 1$  to  $0$  do
3:     if  $i = h - 1$  then
4:        $A_{k_0, \dots, k_i}[k_{i+1} - k_i] = A_{k_0, \dots, k_i}[0] \oplus A_{k_0, \dots, k_{i+1}}[0]$ 
5:     else if  $i \leq h - 2$  and  $i = d - 1$  then
6:        $A_{k_0, \dots, k_i}[k_{i+1} - k_i] = A_{k_0, \dots, k_i}[k_{i+2} - k_i] \oplus A_{k_0, \dots, k_{i+1}}[0]$ 
7:     else if  $i \leq h - 2$  and  $i \neq d - 1$  then
8:        $A_{k_0, \dots, k_i}[k_{i+1} - k_i] = A_{k_0, \dots, k_i}[k_{i+2} - k_i] \oplus A_{k_0, \dots, k_{i+1}}[k_{i+2} - k_{i+1}]$ 
9:     end if
10:  end for
11:  return  $A_0[k_1]$ 
12: end procedure

```

Algorithm 2 Enumerate $(k_h, \dots, k_1)$ for inputs in $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$

```

1: //  $H[0]$  stores  $\text{Hw}(x_{n_1}, \dots, x_1)$ ,  $H[i]$  stores  $\text{Hw}(x_{\sum_{j=1}^{i+1} n_j}, \dots, x_{1+\sum_{j=1}^i n_j})$ .
2: //  $K[h - i + 1]$  stores  $k_i$  for  $1 \leq i \leq h$ , i.e.,  $(k_h, \dots, k_1) = (K[1], \dots, K[h])$ .
3: //  $\text{num} = I[i]$  represents that  $i \in [1 + \sum_{j=1}^{\text{num}} n_j, \sum_{j=1}^{\text{num}+1} n_j]$ .
4: procedure ENU( $K, H, I, W, N, h, \text{end}$ )
5:   for all  $i$  in  $[1, \text{end}]$  do
6:      $K[h + 1] = i$ ,  $\text{num} = I[i]$ 
7:      $H[\text{num}] = H[\text{num}] + 1$  // increase the Hamming weight
8:     Output  $h + 1$  and the necessary (at most  $d + 1$ ) indices from  $K$ 
9:     if  $H[\text{num}] < W[\text{num}]$  then
10:      ENU( $K, H, I, W, N, h + 1, K[h + 1] - 1$ )
11:     else if  $H[\text{num}] = W[\text{num}]$  and  $\text{num} > 0$  then
12:      ENU( $K, H, I, W, N, h + 1, N[\text{num} - 1]$ )
13:     else
14:       $H[\text{num}] = H[\text{num}] - 1$  // reset the Hamming weight
15:     end if
16:   end for
17:   if  $h > 0$  then
18:      $\text{num} = I[K[h]]$ ,  $H[\text{num}] = H[\text{num}] - 1$  //backtracking
19:   end if
20: end procedure
21:
22: procedure MAIN
23:    $W[] = \{w_1, \dots, w_s\}$ ,  $N[] = \{n_1, n_1 + n_2, \dots, \sum_{i=1}^s n_i\}$ 
24:    $K[n + 1] = \{n + 1, 0, \dots, 0\}$ ,  $H[s] = \{0, \dots, 0\}$ 
25:   Initialize  $I[]$ :  $I[0] = 0$  and  $I[i] = j$  if  $N[j - 1] < i \leq N[j]$  for  $i \in [1, n]$ 
26:   ENU( $K, H, I, W, N, 0, n$ )
27: end procedure

```

Definition 3. The support of a binary vector $(x_n, \dots, x_1)$ denoted by

$$(k_h, \dots, k_1) = \text{Sup}(x_n, \dots, x_1) \text{ where } k_1 < \dots < k_h$$

is defined as the set of nonzero positions counted from 1. Specifically,

$$\forall i \in \{k_h, \dots, k_1\} : x_i = 1; \forall i \in \{n, \dots, 1\} \setminus \{k_h, \dots, k_1\} : x_i = 0$$

Evaluation phase. Enumerate all vectors in $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ in ascending lex order. In other words, if $\mathbf{x} \in P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ is enumerated after $\mathbf{y} \in P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$, there is $\mathbf{y} <_{\text{lex}} \mathbf{x}$. The evaluation phase procedure is as follows:

Step 1: Get $f(\mathbf{0}^n)$ from $A_0[0]$.

Step 2: Enumerate $(x_n, \dots, x_1) \in P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ in ascending lex order and we start from the second element. For each $(x_n, \dots, x_1)$, compute

$$h = \text{Hw}(x_n, \dots, x_1), (k_h, \dots, k_1) = \text{Sup}(x_n, \dots, x_1)$$

and move to Step 3.

Step 3: Output $f(\mathbf{x}) = \text{EVA}(h, d, k_h, \dots, k_1)$ (Algorithm 1).

As a proof of concept, the code of our algorithm for $d \in \{2, 3\}$ is available at <https://github.com/LFKOKAMI/polynomialEvaluation>.

3.1 Complexity Analysis

We study the memory and time complexity of our algorithm.

Memory complexity. At the initialization phase, there are $\binom{n}{i}$ different Type- i arrays where $0 \leq i \leq d$. Therefore, there are in total $\left(\sum_{i=0}^d \binom{n}{i}\right)$ arrays. The size of A_0 is $1 + n = \binom{n}{0} + \binom{n}{1}$ bits, and the total size of $A_{0,1}, \dots, A_{0,n}$ is $n + \sum_{j=1}^n (n - j) = \binom{n}{1} + \binom{n}{2}$ bits. For each $2 \leq i < d$, the memory required for all Type- i arrays is

$$\begin{aligned} \sum_{j=i}^n \binom{j-1}{i-1} (n+1-j) &= (n+1) \cdot \sum_{j=i}^n \binom{j-1}{i-1} - \sum_{j=i}^n \binom{j-1}{i-1} j \\ &= (n+1) \binom{n}{i} - i \cdot \sum_{j=i}^n \binom{j}{i} = (n+1) \binom{n}{i} - i \cdot \binom{n+1}{i+1} \\ &= \binom{n+1}{i+1} = \binom{n}{i} + \binom{n}{i+1} \end{aligned}$$

bits, since the number of Type- i arrays of size $n+1-j$ is $\binom{j-1}{i-1}$, i.e., there are in total $\binom{j-1}{i-1}$ possible combinations of $k_1, \dots, k_{i-1}$ satisfying $1 \leq k_1 < k_2 < \dots < k_{i-1} < j = k_i$. The size of all Type- d arrays is simply $\binom{n}{d}$ bits. Hence, the memory complexity of our algorithm is

$$\binom{n}{d} + \sum_{i=0}^{d-1} \binom{n}{i} + \binom{n}{i+1} = \binom{n}{\leq d} + \binom{n}{\leq d} - 1 < 2 \cdot \binom{n}{\leq d}.$$

Time complexity. The time is mainly consumed to update the arrays for each input $(x_n, \dots, x_1)$, which corresponds to at most d xor operations (see Algorithm 1). In addition, we need to initialize $\binom{n}{\leq d}$ entries of the arrays at the initialization phase. Therefore, the number of bit operations of our algorithm is estimated as

$$\binom{n}{\leq d} + d \cdot \prod_{i=1}^s \binom{n_i}{\leq w_i}.$$

When $s = 1, n_1 = w_1 = n$, we get the time complexity $\binom{n}{\leq d} + d \cdot 2^n$.

3.2 Minor Issues in Computing Indices

One may observe that in our algorithm, it is necessary to compute

$$\begin{aligned} h &= \text{Hw}(x_n, \dots, x_1), (k_h, \dots, k_1) = \text{Sup}(x_n, \dots, x_1), \\ k_{i+1} - k_i, k_{i+2} - k_i &\text{ for } i \in [0, d-1]. \end{aligned}$$

Moreover, we can find that these are only related to the input $(x_n, \dots, x_1)$ and the degree of f . Hence, if there are multiple polynomials of degree d for evaluations over the same structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$, the cost to compute these can be amortized, as they only need to be computed once and then can be shared for polynomial evaluations of different polynomials. Formally speaking, assuming that there are m polynomials of degree d for evaluations over the space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$, the total time complexity is

$$T_{\text{index}} + m \cdot \left(\binom{n}{\leq d} + d \cdot \prod_{i=1}^s \binom{n_i}{\leq w_i} \right)$$

where T_{index} denotes the cost to compute these indices and is in $\mathcal{O}(\prod_{i=1}^s \binom{n_i}{\leq w_i})$, which is independent of m . We emphasize that it is a common setting to evaluate multiple polynomials of the same degree in the same space, which frequently appears in solving systems of Boolean equations like [3, 4].

Consequently, we believe that computing these indices is a minor issue. Indeed, the same issue to compute indices $(k_d, \dots, k_1)$ also appears in all derivative-based algorithms [2, 3, 7], and such a cost is not taken into account.

However, for completeness, we also give a recursive algorithm to make T_{index} as small as possible, as shown in Algorithm 2. As we target a non-full space, it is indeed a bit technical to enumerate all $(k_h, \dots, k_1)$ whose corresponding binary vectors are in ascending lex order. But the recursive algorithm is still not that difficult.

Complexity of Algorithm 2. As there are in total $\prod_{i=1}^s \binom{n_i}{\leq w_i} - 1$ nonzero inputs, Line (6, 7) will be performed $\prod_{i=1}^s \binom{n_i}{\leq w_i} - 1$ times, where 2 additions are performed⁷ each time. In addition, the total number of times that Line (14, 18)

⁷As can be observed, the arithmetic operation in the algorithm is either adding 1 or subtracting 1, we can avoid these operations by using two precomputed tables of size n storing $i+1$ ($0 \leq i < n$) and $i-1$ ($1 \leq i \leq n$), respectively.

is performed is also $\Pi_{i=1}^s \binom{n_i}{\leq w_i} - 1$, where 1 addition is performed each time. Moreover, at Line 8, we get the corresponding $(k_h, \dots, k_1)$ and compute necessary indices $k_{i+1} - k_i, k_{i+2} - k_i$ for $i \in [0, d-1]$, which takes at most $2d$ operations. Therefore, we estimate T_{index} as

$$T_{\text{index}} = 3 \cdot \Pi_{i=1}^s \binom{n_i}{\leq w_i} + 2d \cdot \Pi_{i=1}^s \binom{n_i}{\leq w_i}.$$

Thus, when $m \geq 4$, the cost of computing indices can be almost amortized.

3.3 Applications

Solving the open problem in [2]. Setting $s = 1$, $n_1 = n$, $w_1 < n$, we directly solved the open problem raised in [2].

Application to Dinur's polynomial method [4]. There are two critical steps in Dinur's polynomial method:

Step 1: Enumerate all solutions of a system of $n_1 + 1$ equations $f_1(x_n, \dots, x_1) = 0, \dots, f_{n_1+1}(x_n, \dots, x_1) = 0$ of degree d in the space $P_{n-n_1, n_1}^{w_2, n_1}$ where

$$w_2 = d \cdot (n_1 + 1) - n_1 + 1.$$

Step 2: After Step 1, $n_1 + 1$ different polynomials in $n - n_1$ variables of degree upper bounded by w_2 are constructed with the polynomial interpolation. Then, perform polynomial evaluations for these $n_1 + 1$ polynomials over the space $P_{n-n_1}^{n-n_1} = \{0, 1\}^{n-n_1}$.

More technical details of Dinur's polynomial method can be referred to [4, 9].

For Step 1, Dinur observed that the least significant n_1 bits can be traversed using Gray code. Then, the Gray code enumeration can be tweaked in such a way that in every 2^{n_1} steps, it updates to the next binary vector $x' \in P_{n-n_1, n_1}^{w_2, 0}$ that needs to be enumerated. As x' differs in at most n bits from any previous inputs $x \in \mathbb{F}_2^n$, Dinur claimed that computing $f_i(x')$ takes at most n steps of the FES algorithm⁸, and each step of the algorithm takes at most d XOR operations. Hence, computing $f_i(x')$ takes at most $n \cdot d$ bit operations. Such a procedure is executed every 2^{n_1} steps. Hence, if $n \ll 2^{n_1}$, evaluating $f_i(x)$ over $P_{n-n_1, n_1}^{w_2, n_1}$ takes $d \cdot \binom{n-n_1}{\leq w_2} \cdot 2^{n_1}$ bit operations. In addition, Dinur left an interesting question to adapt the FES algorithm with monotone Gray code to solve Step 1.

However, in the original FES algorithm [3] to evaluate f of degree d over $\mathbb{F}_2^n$ with Gray code, there is never a conclusion that computing $f(x \oplus e_i)$ from $f(x)$ takes at most d bit operations for any (x, e_i) , which will be clear in Section 4.

⁸We can interpret it in the way that the algorithm first computes $f_i(x \oplus e_{k_1})$, then computes $f_i(x \oplus \sum_{j=1}^2 e_{k_j})$, and continue this for at most n steps such that it finally reaches to computing $f_i(x \oplus \sum_{j=1}^u e_{k_j})$ where $x' = x \oplus \sum_{j=1}^u e_{k_j}$.

Hence, it is unclear whether such a tweaked FES algorithm can run as expected, given the fact that the correctness of this algorithm is not proved and how it actually runs is not explicitly described in [4].

Based on the tweaked FES algorithm, which works in time $d \cdot \binom{n-n_1}{\leq w_2} \cdot 2^{n_1}$, the time complexity (T_{step1}) and memory complexity M_{step1} of Step 1 are computed as follows in [4]:

$$T_{\text{step1}} = 2d \cdot \log n \cdot 2^{n_1} \cdot \binom{n-n_1}{\leq w_2}, \quad M_{\text{step1}} = (n_1 + 1) \cdot \binom{n}{\leq d}.$$

As the correctness of the tweaked FES algorithm is not proved, we suggest using our algorithm to directly solve Step 1 of Dinur's polynomial method by setting $s = 2, n_2 = n - n_1, w_1 = n_1$. In this way, the time complexity of Step 1 remains unchanged, while the memory complexity M'_{step1} is only doubled, i.e., $M'_{\text{step1}} = (n_1 + 1) \cdot 2 \cdot \binom{n}{\leq d}$.

For Step 2, Dinur used his memory-efficient Möbius transform, where the time complexity T_{step2} and memory complexity M_{step2} are given below:

$$T_{\text{step2}} = (n_1 + 1) \cdot (n - n_1) \cdot 2^{n-n_1}, \quad M_{\text{step2}} = (n_1 + 1) \cdot 2 \cdot \binom{n-n_1}{\leq w_2}.$$

Our algorithm can reduce the time complexity by a factor of $\frac{n-n_1}{w_2}$, without affecting its memory complexity. As w_2 is relatively large, i.e., $w_2 \approx (n - n_1)/3$, this improvement is incremental. Hence, we do not consider the change of this time complexity.

As a result, to solve a system of Boolean equations in n variables of degree d , Dinur estimated the time complexity T_1 and memory complexity M_1 of the polynomial method as⁹

$$T_1 = 4 \cdot (T_{\text{step1}} + T_{\text{step2}}), \quad M_1 = 4 \cdot \left((n_1 + 1) \cdot \binom{n}{\leq d} + M_{\text{step2}} \right).$$

With our new memory-efficient algorithm for Step 1, the new time complexity T_2 and memory complexity M_2 are as follows:

$$T_2 = 4 \cdot (T_{\text{step1}} + T_{\text{step2}}), \quad M_2 = 4 \cdot \left((n_1 + 1) \cdot 2 \cdot \binom{n}{\leq d} + M_{\text{step2}} \right).$$

Indeed, the total memory complexity is dominated by M_{step2} . As shown in [4], the asymptotical memory complexity is $\mathcal{O}(n^3 \cdot 2^{n-2n_1})$ and $n_1 < n/5$. For most relevant settings for (n, d) , we have $(n_1 + 1) \cdot 2 \cdot \binom{n}{\leq d} < n \cdot \binom{n}{\leq d} < n^3 \cdot 2^{n-2n_1}$. Indeed, polynomial method can significantly beat the fast exhaustive search only for small d , as shown in [4]. In other words, by using our new algorithm to solve Step 1, the time complexity and memory complexity given in [4] can remain unchanged.

⁹To efficiently filter wrong solutions, this is performed in parallel for 4 different systems of $n_1 + 1$ equations.

Application to the setting $P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = \mathbb{F}_2^n$. It is clear that by setting $s = 1, n_1 = w_1 = n$, our algorithm can evaluate $f(x)$ of degree d over $\mathbb{F}_2^n$ with $\binom{n}{\leq d} + d \cdot 2^n$ bit operations and $2 \cdot \binom{n}{\leq d}$ bits of memory. As shown in Table 1, our algorithm beats all state-of-the-art memory-efficient algorithms [2–4, 7], mainly thanks to an extremely simple yet efficient initialization phase. Moreover, we emphasize that the main advantage of our algorithm lies in its applicability to a more general structured input space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

Table 1: Comparison between different algorithms for the evaluation of a degree- d Boolean polynomial in n variables on all inputs over $\mathbb{F}_2^n$

Method	Time (T)	Memory
standard Möbius transform	$n \cdot 2^n$	2^n
fast exhaustive search [3]	$\binom{n}{\leq d}^2 + d \cdot 2^n$	$\binom{n}{\leq d}$
fast enumeration algorithm [7]	$\binom{n}{\leq d}^2 + d \cdot 2^n$	$\log n \cdot n \cdot \binom{n}{\leq d}$
memory-efficient Möbius transform [4]	$2^{n-1} + d \cdot 2^n < T < n \cdot 2^n$	$n \cdot \binom{n}{\leq d}$
memory-efficient Möbius transform [4]	$n \cdot 2^n$	$2 \cdot \binom{n}{\leq d}$
memory-efficient Möbius transform [2]	$2d \cdot 2^n$	$\binom{n}{\leq d}$
our algorithm	$\binom{n}{\leq d} + d \cdot 2^n$	$2 \cdot \binom{n}{\leq d}$

4 Revisiting Fast Exhaustive Search Framework

From this section, we start discussing the correctness of our algorithm. Our algorithm is inspired by [3, 7] and both techniques in [3, 7] are indeed based on the same framework in [3] to perform efficient polynomial evaluations, i.e., the derivative-based framework. In this section, we rephrase this framework [3] and give some insight that will be closely related to our algorithm.

Derivative of a Boolean polynomial. We always study a Boolean polynomial $f(x)$ in the following sections, whose ANF is specified in Equation 1. For such a polynomial f and a non-empty set $\mathcal{I} \subseteq \{1, \dots, n\}$, it can always be written as

$$f(x_n, \dots, x_1) = \left(\prod_{i \in \mathcal{I}} x_i \right) \cdot D_{\mathcal{I}}(f)(x_n, \dots, x_1) \oplus f'(x_n, \dots, x_1) \quad (3)$$

where the monomial $\prod_{i \in \mathcal{I}} x_i$ does not divide any monomial in $f'(x_n, \dots, x_1)$. This form has been extensively exploited in the so-called cube attack [5] and the polynomial method [4]. Although the variable x_i for any $i \in \mathcal{I}$ will not appear in the polynomial $D_{\mathcal{I}}(f)(x_n, \dots, x_1)$, for convenience, we still write this

polynomial in terms of $(x_n, \dots, x_1)$. Note that each coefficient of the monomial in terms of x_i in $D_{\mathcal{I}}(f)(x_n, \dots, x_1)$ is 0 for $i \in \mathcal{I}$.

Let $\mathcal{I} = \{k_1, \dots, k_i\}$. It is well-known that

$$D_{\mathcal{I}}(f)(x_n, \dots, x_1) = \sum_{(x_{k_i}, \dots, x_{k_1}) \in \mathbb{F}_2^i} f(x_n, \dots, x_1).$$

Consequently, we have

$$\begin{cases} D_{\{j\} \cup \mathcal{I}}(f) = D_{\{j\}}(D_{\mathcal{I}}(f)), \forall j \notin \mathcal{I} \\ D_{\mathcal{I}}(f)(\mathbf{0}^n) = \mu(\sum_{j \in \mathcal{I}} e_j) \end{cases} \quad (4)$$

Indeed, $D_{\mathcal{I}}(f)$ is the derivative of f with respect to the variables $x_{k_1}, \dots, x_{k_i}$.

Note that $k_0 = 0$ in this paper. For convenience, throughout this paper, we also sometimes use $D_{k_0}(f)$ (or $D_0(f)$) to denote f , and $D_{\mathcal{I}}$ is written as $D_{k_0, \dots, k_i}$ when $\mathcal{I} = \{k_1, \dots, k_i\}$. Sometimes, we also simply write it as $D_{k_1, \dots, k_i}$.

4.1 Rephrasing the Framework for Polynomial Evaluation

For a Boolean vector $\mathbf{x} = (x_n, \dots, x_1)$, we define

$$\begin{aligned} \mathbf{x}_{(k_i)} &= \mathbf{x} \oplus \mathbf{e}_{k_i}, \mathbf{x}^{(k_i)} = \mathbf{x} \wedge \overline{\mathbf{e}_{k_i}}, \text{ for } 1 \leq k_i \leq n, \\ \mathbf{x}^{(0)} &= \mathbf{x}, \mathbf{x}^{(0, k_1, \dots, k_i)} = \mathbf{x} \wedge \overline{\mathbf{e}_{k_i}} \wedge \dots \wedge \overline{\mathbf{e}_{k_1}}, \\ \mathbf{x}_{(k_j)}^{(0, k_1, \dots, k_i)} &= (\mathbf{x} \oplus \mathbf{e}_{k_j}) \wedge (\overline{\mathbf{e}_{k_i}} \wedge \dots \wedge \overline{\mathbf{e}_{k_1}}). \end{aligned}$$

Specifically, $\mathbf{x}_{(k_i)}$ represents a new vector obtained by flipping the k_i -th bit of $\mathbf{x}$, while $\mathbf{x}^{(k_i)}$ represents a new vector obtained by setting the k_i -th bit of $\mathbf{x}$ to 0. For example, if $\mathbf{x} = (0, 0, 1, 1)$, then $\mathbf{x}_{(0, 2, 3)} = (0, 1, 0, 1)$ and $\mathbf{x}^{(0, 2)} = (0, 0, 0, 1)$.

Both memory-efficient algorithms [3, 7] are based on the relation of the following recursive form to evaluate a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d in the full space $\mathbb{F}_2^n$:

$$\forall i \in [1, n] : f(\mathbf{x}) = f(\mathbf{x}_{(i)}) \oplus D_i(f)(\mathbf{x}^{(i)}), \quad (5)$$

As $D_i(f)$ is again a polynomial of degree $d - 1$, the relation in Equation 5 can be used recursively by replacing the polynomial f and the input $\mathbf{x}$ with $D_i(f)$ and $\mathbf{x}^{(i)}$, respectively. Finally, the problem is reduced to evaluate a constant polynomial. In this way, $f(x_n, \dots, x_1)$ can be computed for any input in $\mathbb{F}_2^n$.

Although the idea implied in such a recursive relation in Equation 5 is simple, some deeper reasons for how and why this can be used might be overlooked. To address this issue, we first convert this recursive description into a non-recursive form.

Converting recursive form into non-recursive form. To evaluate $f(\mathbf{x})$, based on Equation 4, the recursive form in Equation 5 can be rephrased in the way that $\exists \ell : [1, d]$ and $\{k_1, \dots, k_\ell\} \subseteq \{1, \dots, n\}$, and we have

$$\left\{ \begin{array}{l} D_{k_0, \dots, k_\ell}(f)(\mathbf{x}^{(k_0, \dots, k_\ell)}) \text{ can be efficiently computed} \\ D_{k_0, \dots, k_{\ell-1}}(f)(\mathbf{x}^{(k_0, \dots, k_{\ell-1})}) = D_{k_0, \dots, k_{\ell-1}}(f)(\mathbf{x}_{(k_\ell)}^{(k_0, \dots, k_{\ell-1})}) \oplus D_{k_0, \dots, k_\ell}(f)(\mathbf{x}^{(k_0, \dots, k_\ell)}) \\ \dots \\ D_{k_0, \dots, k_i}(f)(\mathbf{x}^{(k_0, \dots, k_i)}) = D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) \oplus D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}^{(k_0, \dots, k_{i+1})}) \\ \dots \\ D_{k_0}(f)(\mathbf{x}^{(k_0)}) = D_{k_0}(f)(\mathbf{x}_{(k_1)}^{(k_0)}) \oplus D_{k_0, k_1}(f)(\mathbf{x}^{(k_0, k_1)}) \end{array} \right. \quad (6)$$

Note that $k_0 = 0, \mathbf{x}^{(k_0)} = \mathbf{x}^{(0)} = \mathbf{x}$ and $D_{k_0}(f) = D_0(f) = f$. Also, note that $k_i \neq k_j$ for $i \neq j$ by definition, but the order inside $k_1, \dots, k_\ell$ is not specified, i.e., it is not necessarily $k_1 < \dots < k_\ell$. Moreover, the coefficient of any monomial in terms of x_j in $D_{k_0, \dots, k_i}(f)$ is zero when $j \in \{k_1, \dots, k_i\}$.

Terminating conditions. The recursion terminates when $\ell = d$ or $\mathbf{x}^{(k_0, \dots, k_\ell)} = \mathbf{0}^n$, since

$$\ell = d \Rightarrow \forall \mathbf{x} \in \mathbb{F}_2^n : D_{k_0, \dots, k_d}(f)(\mathbf{x}^{(k_0, \dots, k_d)}) = \mu\left(\sum_{i=1}^d \mathbf{e}_{k_i}\right),$$

$$\mathbf{x}^{(k_0, \dots, k_\ell)} = \mathbf{0}^n \Rightarrow \forall \{k_1, \dots, k_\ell\} \subseteq \{1, \dots, n\} : D_{k_0, \dots, k_\ell}(f)(\mathbf{0}^n) = \mu\left(\sum_{i=1}^{\ell} \mathbf{e}_{k_i}\right).$$

Note that $\mu(\sum_{i=1}^d \mathbf{e}_{k_i})$ is the coefficient of the monomial $x_{k_1} x_{k_2} \dots x_{k_d}$ in f , and $\mu(\sum_{i=1}^{\ell} \mathbf{e}_{k_i})$ is the coefficient of the monomial $x_{k_1} x_{k_2} \dots x_{k_\ell}$ in f .

Based on this non-recursive form, we can derive three critical implications for this framework to work.

Implication on evaluation rule. The first implication is that there should be a rule to determine the value of $(k_1, \dots, k_\ell)$ for any given input $\mathbf{x}$. Specifically,

- For any $i \in [0, d-1]$, given the input $\mathbf{x}^{(k_0, \dots, k_i)}$ to $D_{k_0, \dots, k_i}(f)$, there is a rule to determine a unique $\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}$, i.e., k_{i+1} is uniquely determined for each input $\mathbf{x}^{(k_0, \dots, k_i)}$.

For simplicity, we define the concept called **evaluation rule**.

Definition 4 (Evaluation Rule). The evaluation rule $\mathcal{R}$ for evaluating a polynomial $g(x_n, \dots, x_1)$ over an ordered input set S is defined in the way that given an input $\mathbf{x} \in S$ to the polynomial g , there is a unique choice of $\mathbf{x}' \in S$ such that we compute $g(\mathbf{x})$ with

$$g(\mathbf{x}) = g(\mathbf{x}') \oplus D_i(g)(\mathbf{x}^{(i)}) \text{ where } \mathbf{x} \oplus \mathbf{x}' = \mathbf{e}_i.$$

In particular, this is denoted by $\mathbf{x}' \xrightarrow{\mathcal{R}} \mathbf{x}$.

Algorithm 3 Find ordered input set based on the evaluation rule $\mathcal{R}$ to evaluate a polynomial over S

```

1: procedure NEXTSET( $S, \mathcal{R}, i$ )
2:    $S' = \emptyset$  // empty set
3:   for  $j \in [1, |S| - 1]$  do
4:     Find  $j' \in [0, j - 1]$  such that  $S[j'] \xrightarrow{\mathcal{R}} S[j]$  and  $e_i = S[j'] \oplus S[j]$ 
5:     Add  $(S[j])^{(i)}$  to the set  $S'$ 
6:   end for
7:   return  $S'$ 
8: end procedure

```

Implication on evaluation order. The second implication is related to the order to enumerate the input x of f . According to the non-recursive description, we have the following conditions:

- For any $i \in [0, d - 1]$, the input $x^{(k_0, \dots, k_i)}$ to $D_{k_0, \dots, k_i}(f)$ must be enumerated after the input $x^{(k_0, \dots, k_i)}_{(k_{i+1})}$, and $D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)}_{(k_{i+1})})$ must be stored somewhere at the point to compute $D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)})$. In this way, computing $D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)})$ only requires to compute $D_{k_0, \dots, k_{i+1}}(f)(x^{(k_0, \dots, k_{i+1})})$.

For simplicity, we define the concept called **evaluation order**.

Definition 5 (Evaluation Order). If the evaluation of a polynomial g on the input x is performed before the evaluation of g on the input x' , we say $x <_g x'$.

For convenience, throughout this paper, $S_{k_0, \dots, k_i}$ is used to denote the ordered input set for $D_{k_0, \dots, k_i}(f)$, and the evaluation rule to evaluate $D_{k_0, \dots, k_i}(f)$ over $S_{k_0, \dots, k_i}$ is denoted by $\mathcal{R}_{k_0, \dots, k_i}$. In other words, we have

$$\forall j \in [1, |S_{k_0, \dots, k_i}| - 1] : S_{k_0, \dots, k_i}[j - 1] <_{D_{k_0, \dots, k_i}(f)} S_{k_0, \dots, k_i}[j].$$

Moreover, based on Algorithm 3, we have

$$\forall i \in [0, d - 1] : S_{k_0, \dots, k_{i+1}} = \text{NEXTSET}(S_{k_0, \dots, k_i}, \mathcal{R}_{k_0, \dots, k_i}, k_{i+1}). \quad (7)$$

In this way, we can implicitly construct the ordered input sets for all possible polynomials appearing in the procedure to evaluate f on all inputs in S_0 .

Implication on initialization phase. After determining the evaluation order and the evaluation rules to perform polynomial evaluations, we can always implicitly construct the ordered input set for each possible polynomial that will appear in Equation 6. If we reach the point to evaluate the corresponding polynomial on the first element in its input set, we can no longer use Equation 6. As a result, it is required to evaluate each polynomial on the first element in its corresponding ordered input set at the initialization phase, and store these

results in a table. In other words, at the initialization phase, the following set should be computed and stored:

$$S_{\text{ini}} = \{D_{k_0, \dots, k_i}(f)(S_{k_0, \dots, k_i}[0]) \mid i \in [0, d], k_0 = 0, \{k_1, \dots, k_i\} \subseteq \{1, \dots, n\}\}.$$

4.2 Sufficient Condition for the Framework

We have proposed two concepts called **evaluation rule** and **evaluation order** to rephrase the framework for polynomial evaluations proposed in [3]. In this section, we further dive into the rephrased framework and give a sufficient condition to make the framework work efficiently. This sufficient condition will be extremely useful for applying this framework to the structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

Theorem 1 (Sufficient condition). *When evaluating a polynomial $f(x_n, \dots, x_1)$ of degree d over an ordered set S_0 based on Equation 6, let g be the non-constant polynomial that will appear in Equation 6, and the ordered input set for g be S . Moreover, the evaluation rule to evaluate g over S is denoted by $\mathcal{R}$, i.e.,*

$$(g, S, \mathcal{R}) \in \{(D_0(f), S_0, \mathcal{R}_0), \dots, (D_{k_0, \dots, k_{d-1}}(f), S_{k_0, \dots, k_{d-1}}, \mathcal{R}_{k_0, \dots, k_{d-1}})\}.$$

For any such $(g, S, \mathcal{R})$, if

$$\forall x \in S \text{ and } x \neq S[0] : \exists x' \in S, x' <_g x, x' \xrightarrow{\mathcal{R}} x, \quad (8)$$

evaluating f over S_0 requires at most $d \cdot |S_0|$ bit operations, in addition to the cost to evaluate all possible $g(S[0])$ at the initialization phase.

Proof. We show a memory-inefficient algorithm consisting of the initialization phase and evaluation phase to evaluate f over S_0 .

At the initialization phase, compute all possible $g(S[0])$, and add the result $(S[0], g(S[0]))$ to the table TAB_{pre} . Then, start the evaluation phase.

At the evaluation phase, we will store each $(x, g(x))$ in the table TAB_{eva} for any $x \in S$ after $g(x)$ is computed. That is why it is a memory-inefficient algorithm. The procedure to compute $f(x)$ for each $x \in S_0$ is as follows:

1. If $x = S_0[0]$, directly get $f(x)$ from TAB_{pre} and add $(x, f(x))$ to the table TAB_{eva} .
2. If $x \neq S_0[0]$, according to the condition in Equation 8, there will be

$$\exists x' \in S_0 : x' <_f x, x' \xrightarrow{\mathcal{R}_0} x \Rightarrow \exists k_1 \in [1, n] : x' \oplus x = e_{k_1}$$

Hence,

$$f(x) = D_0(f)(x^{(0)}) = D_{k_0}(f)(x_{(k_1)}^{(k_0)}) \oplus D_{k_0, k_1}(f)(x^{(k_0, k_1)})$$

is used to compute $f(x)$ where $D_{k_0}(f)(x_{(k_1)}^{(k_0)})$ can be directly obtained from TAB_{eva} . Consequently, the problem becomes computing $D_{k_0, k_1}(f)(x^{(k_0, k_1)})$. Recursively, $D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)})$ is computed as follows:

- If $\mathbf{x}^{(k_0, \dots, k_i)} = S_{k_0, \dots, k_i}[0]$, get $D_{k_0, \dots, k_i}(f)(\mathbf{x}^{(k_0, \dots, k_i)})$ from TAB_{pre} and add $(\mathbf{x}^{(k_0, \dots, k_i)}, D_{k_0, \dots, k_i}(f)(\mathbf{x}^{(k_0, \dots, k_i)}))$ to TAB_{eva} .
- If $\mathbf{x}^{(k_0, \dots, k_i)} \neq S_{k_0, \dots, k_i}[0]$, according to Equation 8, there exists $\mathbf{x}' \in S_{k_0, \dots, k_i}$ such that

$$\mathbf{x}' <_{D_{k_0, \dots, k_i}(f)} \mathbf{x}^{(k_0, \dots, k_i)}, \mathbf{x}' \xrightarrow{\mathcal{R}_{k_0, \dots, k_i}} \mathbf{x}^{(k_0, \dots, k_i)}, \mathbf{x}' \oplus \mathbf{x}^{(k_0, \dots, k_i)} = \mathbf{e}_{k_{i+1}}.$$

Since $\mathbf{x}' = \mathbf{x}^{(k_0, \dots, k_i)} \oplus \mathbf{e}_{k_{i+1}} = \mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}$, the relation

$$D_{k_0, \dots, k_i}(f)(\mathbf{x}^{(k_0, \dots, k_i)}) = D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) \oplus D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}^{(k_0, \dots, k_{i+1})})$$

is used to compute $D_{k_0, \dots, k_i}(f)(\mathbf{x}^{(k_0, \dots, k_i)})$ where $D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)})$ can be directly obtained from TAB_{eva} . The problem is then reduced to computing $D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}^{(k_0, \dots, k_{i+1})})$.

The recursion must end at a certain point, i.e., it will start backtracking when $(k_1, \dots, k_{i+1})$ satisfies the following conditions:

$$\mathbf{x}^{(k_0, \dots, k_{i+1})} = \mathbf{0}^n \text{ or } i+1 = d.$$

This has been explained by the terminating conditions.

Based on the above procedure, computing $f(x)$ for any $x \in S_0$ takes at most d bit operations. Consequently, evaluating f over S_0 takes at most $d \cdot |S_0|$ bit operations for the evaluation phase. $\square$

Comparison. We emphasize that the same idea in the proof of Theorem 1 has been implicitly used in the fast exhaustive search framework [3] to evaluate a polynomial on all inputs in $\mathbb{F}_2^n$ using Gray code. Here, we make a further generalization that can possibly lead to broader applications. Specifically, neither the evaluation orders, nor the evaluation rules, nor even the input set S_0 that might not be the full space $\mathbb{F}_2^n$, are specified. Therefore, they can be customized for concrete applications as long as the sufficient condition can hold, e.g., for efficiently evaluating a Boolean polynomial over the structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

4.3 Impact of the Sufficient Condition

Based on the sufficient condition, we prove that it is either infeasible or non-trivial to use monotone Gray code in the FES algorithm [7].

Let the evaluation rule $\mathcal{R}_{\text{GC}}$ over an ordered set S be defined as

$$\forall j \in [1, |S| - 1] : S[j - 1] \xrightarrow{\mathcal{R}_{\text{GC}}} S[j].$$

Lemma 2. [3] Let $S_0 = G_n$ be an ordered set of size 2^n , where G_n is the Gray code for all $x \in \mathbb{F}_2^n$. For any ordered set $S_{k_0, \dots, k_i}$ obtained with

$$S_{k_0, \dots, k_i} = \text{NEXTSET}(S_{k_0, \dots, k_{i-1}}, \mathcal{R}_{GC}, k_i),$$

it always satisfies that if $|S_{k_0, \dots, k_i}| > 1$, then.

$$\forall j \in [1, |S_{k_0, \dots, k_i}| - 1] : \text{Hw}(S_{k_0, \dots, k_i}[j-1] \oplus S_{k_0, \dots, k_i}[j]) = 1.$$

Lemma 2 indicates that by setting any evaluation rule $R_{k_0, \dots, k_i}$ as R_{GC} , each ordered input set (of size > 1) of the polynomial $D_{k_0, \dots, k_i}(f)$ has the property that each pair of consecutive elements in the set only differ in 1 bits, thus satisfying the sufficient condition under the evaluation rule R_{GC} . As a result, the original FES algorithm [3] can work for any degree due to this property under the same evaluation rule R_{GC} .

In the following, we show that for $d \geq 2$, this is no longer the case when changing S_0 to $MG_{n,w}$ for any $w \geq 2$. In other words, it becomes difficult to achieve the time complexity $d \cdot \binom{n}{\leq w}$ when using the FES algorithm with monotone Gray code to evaluate f over P_n^w for $w \geq 2$.

Theorem 2. Let $S_0 = MG_{n,w}$ and $S_{0,i} = \text{NEXTSET}(S_0, \mathcal{R}_{GC}, i)$ for $i \in [1, n]$. When $n \geq 4$ and $w \geq 2$, there must exist $S_{0,i}$ such that

$$\forall j \in [1, |S_{0,i}| - 1] : \text{Hw}(S_{0,i}[j-1] \oplus S_{0,i}[j]) \neq 1.$$

Proof. By definition,

$$MG_{n,w} = \{V_{0,1}[0], \dots, V_{0,1}[s_{0,1} - 1], \dots, V_{i,i+1}[0], \dots, V_{w,w+1}[s_{w,w+1} - 1]\}.$$

Due to $\{x \in \mathbb{F}_2^n \mid \text{Hw}(x) = k\} \subseteq V_{k-1,k} \cup V_{k,k+1}$, we have¹⁰ $s_{1,2} = 2(n-1) \geq n+2$ for $n \geq 4$. In addition, we also have

$$s_{0,1} = 2, s_{n-1,n} = 2, s_{k-1,k} + s_{k,k+1} \geq \binom{n}{k}, \sum_{k=0}^{n-1} s_{k,k+1} = 2^n.$$

Therefore, as n increases, there are many $k \in [1, w]$ satisfying $s_{k,k+1} \geq n+2$. Anyway, when $n \geq 4$ and $w \geq 2$ (so that $V_{2,3}$ is also included in $MG_{n,w}$), there exists $k \in [1, w]$ such that $s_{k,k+1} \geq n+2$.

Based on Lemma 1, for any $n+2$ consecutive elements in such an ordered set $V_{k,k+1}$, there must exist at least 2 different pairs of 2 consecutive elements denoted by $(x, y), (x', y') \neq (x, y)$ where $x, y, x', y' \in V_{k,k+1}$ such that

$$\exists i \in [1, n] : x \xrightarrow{\mathcal{R}_{GC}} y, x' \xrightarrow{\mathcal{R}_{GC}} y', x \oplus y = x' \oplus y' = e_i.$$

¹⁰Note that $s_{0,1} = 2$, resulting in $s_{1,2} = 2(n-1)$ or $s_{1,2} = 2(n-1) - 1$. In this case of $s_{1,2} = 2(n-1) - 1$, the last element in $V_{1,2}$ must has Hamming weight 1, and hence the first element in $V_{2,3}$ must has Hamming weight 2. We can thus equivalently treat the first element in $V_{2,3}$ as the last element in $V_{1,2}$, resulting in also $s_{1,2} = 2(n-1)$.

As a result, according to the function NEXTSET in Algorithm 3, $\mathbf{y}^{(i)}$ and $(\mathbf{y}')^{(i)}$ will be added to $S_{0,i}$. Moreover, $\mathbf{y}^{(i)} = \mathbf{y} \wedge \bar{e}_i$ and $(\mathbf{y}')^{(i)} = \mathbf{y}' \wedge \bar{e}_i$ are consecutive in $S_{0,i}$. Based on Lemma 1, $\text{Hw}(\mathbf{y} \wedge \bar{e}_i) = \text{Hw}(\mathbf{y}' \wedge \bar{e}_i) = k$, resulting in $\text{Hw}(\mathbf{y}^{(i)} \oplus (\mathbf{y}')^{(i)}) \neq 1$. $\square$

Theorem 2 indicates that using $\mathcal{R}_{\text{GC}}$ for f , there is at least one polynomial $D_{0,i}(f)$ whose ordered input set $S_{0,i}$ does not preserve the property that any 2 consecutive elements differ in only 1 bit. Hence, $S_{0,i}[j-1] \xrightarrow{\mathcal{R}_{\text{GC}}} S_{0,i}[j]$ does not always hold, i.e., due to $S_{0,i}[j-1] \oplus S_{0,i}[j] \notin \{e_1, \dots, e_n\}$. Indeed, as n increases, there are many (not only 2) consecutive elements in $S_{0,i}$ such that they differ in more than 1 bit. This can also be easily observed from the proof of Lemma 1. Specifically, there are $|V_{k,k+1}|$ consecutive elements that form $|V_{k,k+1}| - 1$ consecutive pairs $(\mathbf{x}, \mathbf{y})$ of 2 consecutive elements in $V_{k,k+1}$, where each $(\mathbf{x}, \mathbf{y})$ satisfies $\text{Hw}(\mathbf{x} \oplus \mathbf{y}) = 1$. Hence, there will be at least $\lceil \frac{|V_{k,k+1}| - 1}{n} \rceil$ consecutive elements in a set $S_{0,i}$ whose Hamming weight is always k . It is also possible that there are many such input sets $S_{0,i}$. All of these are difficult to predict without explicitly constructing each input set for $D_{0,i}(f)$ from $MG_{n,w}$ under the evaluation rule $\mathcal{R}_{\text{GC}}$.

Based on Theorem 2, in order to combine the FES algorithm with monotone Gray code, the same evaluation order $\mathcal{R}_{\text{GC}}$ can no longer be used to evaluate each $D_{k_0, \dots, k_i}(f)$. Hence, we claim that it is either infeasible to use monotone Gray code in the FES algorithm, or it is nontrivial to find a proper evaluation order $\mathcal{R}_{k_0, \dots, k_i}$ for each polynomial $D_{k_0, \dots, k_i}(f)$ that can satisfy the sufficient condition. This is mainly due to the fundamental structural difference between Gray code and monotone Gray code.

5 Polynomial Evaluation over Structured Space

Based on the sufficient condition specified in Theorem 1, to devise an efficient algorithm to evaluate a Boolean polynomial $f(x_n, \dots, x_1)$ of degree d over the space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$, it is sufficient to find the corresponding evaluation rules and evaluation order.

In this section, we prove that the evaluation rule and the evaluation order proposed in Furue-Takagi's FEA algorithm [7] can be reused to perform efficient polynomial evaluations over $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

Remark 1. Although the same evaluation rule and evaluation order as in [7] are used in our algorithm, our algorithm is distinguished from Furue-Takagi's FEA algorithm [7] by the initialization phase, a better data structure to store intermediate results, and simplicity, resulting in improved time complexity and memory complexity under the setting $P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = \mathbb{F}_2^n$. Moreover, we rigorously prove that it is feasible to apply such an evaluation rule and evaluation order to polynomial evaluations over a more general structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1} \subseteq \mathbb{F}_2^n$, which also directly solves an open problem proposed in [2]. All these are made possible due to our interpretation of the fast exhaustive search framework [3],

Theorem 1 developed for its sufficient condition, Lemma 3 described later for testing the sufficient condition, and a strategy to optimize the memory consumption.

The evaluation order and evaluation rule. Let $S_0 = P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$. S_0 is an ordered set satisfying the following order:

$$\forall j \in [1, |S_0| - 1] : S_0[j - 1] <_{\text{lex}} S_0[j].$$

The evaluation rule for any polynomial $D_{k_0, \dots, k_i}(f)$ over $S_{k_0, \dots, k_i}$ is the same. This evaluation rule is denoted by $\mathcal{R}_{\text{lex}}$, which is defined below:

$$(x_n, \dots, x_{j+1}, 0, \underbrace{0, \dots, 0}_{j-1}) \xrightarrow{\mathcal{R}_{\text{lex}}} (x_n, \dots, x_{j+1}, 1, \underbrace{0, \dots, 0}_{j-1}) \quad (9)$$

In other words, to evaluate a polynomial g on the input $\mathbf{x} \in S$, it is necessary to find the smallest index i such that $x_i = 1$, in which case we have $\mathbf{x} = (x_n, \dots, x_{j+1}, 1, \underbrace{0, \dots, 0}_{j-1}) \in S$. Consequently, $\mathbf{x}_{(j)} = (x_n, \dots, x_{j+1}, 0, \underbrace{0, \dots, 0}_{j-1}) \in S$ has to hold, since $g(\mathbf{x})$ is computed based on the following relation under $\mathcal{R}_{\text{lex}}$:

$$g(\mathbf{x}) = g(\mathbf{x}_{(j)}) \oplus D_j(g)(\mathbf{x}^{(j)}).$$

Note that this does not break the evaluation order as

$$\mathbf{x}_{(j)} = (x_n, \dots, x_{j+1}, 0, \underbrace{0, \dots, 0}_{j-1}) <_{\text{lex}} (x_n, \dots, x_{j+1}, 1, \underbrace{0, \dots, 0}_{j-1}) = \mathbf{x}.$$

5.1 Testing Sufficient Condition for Structured Space

We first show an important lemma that plays a crucial role to apply the framework to the structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

Lemma 3. *Let S be an ordered set where*

$$S = P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = P_{n_s}^{w_s} \times \dots \times P_{n_1}^{w_1} \subseteq \mathbb{F}_2^n, |S| > 1, \\ \forall j \in [1, |S| - 1] : S[j - 1] <_{\text{lex}} S[j].$$

If evaluating a Boolean polynomial $g(x_n, \dots, x_1)$ over S under the evaluation rule $\mathcal{R}_{\text{lex}}$ based on Equation 6, there is

$$\forall \mathbf{x} \in S \text{ and } \mathbf{x} \neq S[0] : \exists \mathbf{x}' \in S, \mathbf{x}' <_{\text{lex}} \mathbf{x}, \mathbf{x}' \xrightarrow{\mathcal{R}_{\text{lex}}} \mathbf{x}. \quad (10)$$

Moreover, the input set for $D_i(g)$ is S_i , which is also an ordered set satisfying

$$S_i = P_{n'_q, \dots, n'_1}^{w'_q, \dots, w'_1} \subseteq S \subseteq \mathbb{F}_2^n, \\ \forall j \in [1, |S_i| - 1] : S_i[j - 1] <_{\text{lex}} S_i[j],$$

for some integers $(w'_q, \dots, w'_1, n'_q, \dots, n'_1)$ where $w'_1 = 0, n'_1 = i$ and $\sum_{j=1}^q n'_j = n$.

Proof. For any $x \in S$ and $x \neq S[0] = \mathbf{0}^n$, it can always be written as follows:

$$\begin{cases} \exists k \in [1, n] : x = (x_n, \dots, x_1) = (\underbrace{x_n, \dots, x_{1+n_k+\dots+n_1}}_{\in P_{n_s}^{w_s} \times \dots \times P_{n_{k+1}}^{w_{k+1}}}, \underbrace{v^{n_k}, 0, \dots, 0}_{\sum_{j=1}^{k-1} n_j}), \\ \exists i \in [1 + \sum_{j=1}^{k-1} n_j, \sum_{j=1}^k n_j] : v^{n_k} = (x_{n_k+\dots+n_1}, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1-\sum_{j=1}^{k-1} n_j}) \in P_{n_k}^{w_k} \end{cases} \quad (11)$$

In other words, for any $x \in S$ and $x \neq S[0]$, there is always:

$$\exists i \in [1, n] : x = (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}). \quad (12)$$

Under the given evaluation rule $\mathcal{R}_{\text{lex}}$, to evaluate $g(x)$ where $x \neq S[0]$ satisfies [Equation 12](#), it is always possible to find another element $x' \in S$ satisfying

$$x' = (x_n, \dots, x_{i+1}, 0, \underbrace{0, \dots, 0}_{i-1}) <_{\text{lex}} x, \quad x' \xrightarrow{\mathcal{R}_{\text{lex}}} x = (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}).$$

Here, we only need to prove $x' \in S$ if $x \in S$. As stated above, for any $x \in S$ and $x \neq S[0]$, there exists an integer k satisfying [Equation 11](#). Hence, we have

$$\begin{aligned} u^{n_k} &= (x_{n_k+\dots+n_1}, \dots, x_{i+1}, 0, \underbrace{0, \dots, 0}_{i-1-\sum_{j=1}^{k-1} n_j}) \in P_{n_k}^{w_k}, \\ x' &= (\underbrace{x_n, \dots, x_{1+n_k+\dots+n_1}}_{\in P_{n_s}^{w_s} \times \dots \times P_{n_{k+1}}^{w_{k+1}}}, \underbrace{u^{n_k}, 0, \dots, 0}_{\sum_{j=1}^{k-1} n_j}) \in P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = S. \end{aligned}$$

Due to the above proved property for a single element $x \neq S[0]$ in S , to evaluate $g(x)$ where x satisfies [Equation 12](#) based on [Equation 6](#) under $\mathcal{R}_{\text{lex}}$, the following relation is used:

$$g(x) = g(x_{(i)}) \oplus D_i(f)(x^{(i)}).$$

Hence, $D_i(g)$ is used only for the following inputs to $g(x_n, \dots, x_1)$:

$$\underbrace{(x_n, \dots, x_{1+n_k+\dots+n_1}, x_{n_k+\dots+n_1}, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1-\sum_{j=1}^{k-1} n_j}, \underbrace{0, \dots, 0}_{\sum_{j=1}^{k-1} n_j})}_{\in P_{n_s}^{w_s} \times \dots \times P_{n_{k+1}}^{w_{k+1}}} \in P_{n_k}^{w_k} \in S,$$

which are enumerated in ascending lex order when they are set as inputs to $g(x_n, \dots, x_1)$. Due to the above proved property for a single element $x \neq S[0]$ in

S , the following elements will be set as inputs to $D_i(g)(x_n, \dots, x_1)$ under $\mathcal{R}_{\text{lex}}$:

$$\underbrace{(x_n, \dots, x_{1+n_k+\dots+n_1})}_{\in P_{n_s}^{w_s} \times \dots \times P_{n_{k+1}}^{w_{k+1}}} \underbrace{(x_{n_k+\dots+n_1}, \dots, x_{i+1}, 0, \underbrace{0, \dots, 0}_{i-1-\sum_{j=1}^{k-1} n_j}, \underbrace{0, \dots, 0}_{\sum_{j=1}^{k-1} n_j})}_{\in P_{n_k}^{w_k}} \in S, \quad (13)$$

which are also naturally enumerated in ascending lex order. This proved that the input set of $D_i(f)$ denoted by S_i is an ordered set and

$$\forall j \in [1, |S_i| - 1] : S_i[j - 1] <_{\text{lex}} S_i[j].$$

Moreover, each element in S_i is of the form as in [Equation 13](#). Hence, we have

$$\exists k \in [1, s] : S_i = P_{n_s}^{w_s} \times \dots \times P_{n_{k+1}}^{w_{k+1}} \times P_{n'_k}^{\min(n'_k, w_k)} \times P_i^0,$$

$$n'_k = n_k - (i - \sum_{j=1}^{k-1} n_j).$$

Due to [Equation 13](#), we also have $S_i \subseteq S \subseteq \mathbb{F}_2^n$. $\square$

5.2 Application to Polynomial Evaluations over $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$

Due to [Lemma 3](#), when evaluating a polynomial $f(x_n, \dots, x_1)$ of degree d over the ordered set $S_0 = P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ under $\mathcal{R}_{\text{lex}}$ based on [Equation 6](#), where

$$\forall j \in [1, |S_0| - 1] : S_0[j - 1] <_{\text{lex}} S_0[j],$$

the input set $S_{k_0, \dots, k_i}$ for any polynomial $D_{k_0, \dots, k_i}(f)$ preserves the same structure as $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ with changes in the values of the parameters $(n_s, \dots, n_1, w_s, \dots, w_1)$, and the elements in $S_{k_0, \dots, k_i}$ are listed in ascending lex order. Moreover, according to [Lemma 3](#), we have $S_{k_0, \dots, k_i} \in P_{n'_q, \dots, n'_2, k_i}^{w'_q, \dots, w'_2, 0}$, resulting in $k_1 < k_2 < \dots < k_i$. This is formally stated in [Corollary 1](#).

Corollary 1. *When evaluating a polynomial $f(x_n, \dots, x_1)$ of degree d over the ordered set $S_0 = P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ under $\mathcal{R}_{\text{lex}}$ based on [Equation 6](#), where*

$$\forall j \in [1, |S_0| - 1] : S_0[j - 1] <_{\text{lex}} S_0[j],$$

there will be in total $\binom{n}{\leq d}$ possible polynomials $D_{k_0, \dots, k_i}(f)$ appearing where

$$k_0 = 0, 1 \leq k_1 < \dots < k_i \leq n, i \leq d.$$

Moreover, the input set of $D_{k_0, \dots, k_i}(f)$ denoted by $S_{k_0, \dots, k_i}$ satisfies

$$\exists (n'_q, \dots, n'_2, w'_q, \dots, w'_2) \in \mathbb{N}^{2(q-1)} : S_{k_0, \dots, k_i} = P_{n'_q, \dots, n'_2, k_i}^{w'_q, \dots, w'_2, 0} \subseteq \mathbb{F}_2^n,$$

$$\forall j \in [1, |S_{k_0, \dots, k_i}| - 1] : S_{k_0, \dots, k_i}[j - 1] <_{\text{lex}} S_{k_0, \dots, k_i}[j].$$

Consequently, such an evaluation order (i.e., $<_{\text{lex}}$) and evaluation rule (i.e., $\mathcal{R}_{\text{lex}}$) can make the sufficient condition specified in [Theorem 1](#) hold. In other words, we can safely apply the rephrased fast exhaustive framework stated in [Section 4.1](#) for polynomial evaluations over $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$.

Time complexity of the evaluation phase. For the algorithm given in the proof of Theorem 1, the time complexity of its evaluation phase is upper bounded by $d \cdot |P_{n_s, \dots, n_1}^{w_s, \dots, w_1}|$.

Time complexity of initialization phase. Based on Corollary 1, we have

$$S_{k_0, \dots, k_i}[0] = \mathbf{0}^n.$$

Consequently, at the initialization phase, we only need to evaluate $D_{k_0, \dots, k_i}(f)(\mathbf{0}^n)$ for each $D_{k_0, \dots, k_i}(f)$. As there are $\binom{n}{\leq d}$ possible polynomials $D_{k_0, \dots, k_i}(f)$, i.e., they are

$$D_0(f), D_{(0,1)}(f) \dots, D_{0,n}(f), D_{0,1,2}(f), \dots, D_{0,n-d+1, \dots, n}(f),$$

and $D_{k_0, \dots, k_i}(f)(\mathbf{0}^n) = \mu(\sum_{j=1}^i e_{k_j})$ according to Equation 4, the initialization phase of evaluating f over $P_{n_s, \dots, n_1}^{w_s, \dots, w_1}$ is simply reading all $\binom{n}{\leq d}$ coefficients of f and store them. Hence, for the algorithm given in the proof of Theorem 1 the time complexity of the initialization phase is $\binom{n}{\leq d}$.

Memory complexity. For the algorithm given in the proof of Theorem 1, the size of TAB_{pre} is $\binom{n}{\leq d}$, and the size of TAB_{eva} is upper bounded by $d \cdot |P_{n_s, \dots, n_1}^{w_s, \dots, w_1}|$. Hence, the total memory complexity is $\binom{n}{\leq d} + d \cdot |P_{n_s, \dots, n_1}^{w_s, \dots, w_1}|$.

5.3 Optimizing the Memory Consumption

The algorithm given in the proof of Theorem 1 is close to our algorithm presented at Section 3, where the only difference exists in the memory consumption. In this section, we will address this issue.

As stated above, memory consumption is mainly caused by the table TAB_{eva} , which stores all evaluation results for any polynomial on any input. For convenience, we view TAB_{eva} as the composition of the following $\binom{n}{\leq d}$ tables:

$$TAB_0, TAB_{0,1}, \dots, TAB_{0,n}, \dots, TAB_{0,n-d+1, \dots, n},$$

where $TAB_{k_0, \dots, k_i}$ is used to store the results of evaluating $D_{k_0, \dots, k_i}(f)$ over $S_{k_0, \dots, k_i}$.

In the following, we propose a memory-efficient way to update the table TAB when evaluating the polynomial g on the set S , where (TAB, g, S) is in

$$\{(TAB_0, D_0(f), S_0), \dots, (TAB_{0,n-d+1, \dots, n}, D_{0,n-d+1, \dots, n}(f), S_{0,n-d+1, \dots, n})\}.$$

This memory-efficient way is equivalent to removing redundant elements in TAB timely.

Construction and update of TAB for evaluating g over $S = P_{n'_q, \dots, n'_2, n'_1}^{w'_q, \dots, w'_2, 0}$. According to the algorithm given in the proof of Theorem 1, $g(S[0])$ is stored in TAB_{eva} and thus can be directly obtained when computing $g(S[0])$. The result $(S[0], g(S[0]))$ will be inserted to TAB .

Then, for any $x \in S$ and $x \neq S[0]$ satisfying $x = (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1})$, $g(x)$ is computed using the relation $g(x) = g(x') \oplus D_i(g)(x^{(i)})$ where $x' = (x_n, \dots, x_{i+1}, 0, \underbrace{0, \dots, 0}_{i-1}) \in S$, and $x' \in S$ also satisfies $x' <_g x$. In other words, as long as x is not the first element $S[0]$ in S , before computing $g(x)$, the value $g(x')$ has been computed and stored in TAB .

After such a result $g(x)$ is computed, following the algorithm in the proof of Theorem 1, $(x, g(x))$ will be inserted to TAB . This is a main reason why TAB consumes a lot of memory.

To optimize the memory complexity, we propose Lemma 4 to show that a large number of elements in TAB can be removed after inserting $(x, g(x))$.

Lemma 4. *After inserting $(x, g(x))$ to TAB where $x = (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1})$, all elements $(y, g(y))$ can be removed from TAB if*

$$y \in S_x = \{y \mid y \in S, y = (y_n, \dots, y_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}) <_{\text{lex}} x\}.$$

Proof. To prove this, we only need to show that

$$\forall z \in S \text{ and } x <_{\text{lex}} z : \text{ if } z' \xrightarrow{\mathcal{R}_{\text{lex}}} z, \text{ then } z' \notin S_x. \quad (14)$$

The reason is that only the elements in $\{z \mid z \in S, x <_{\text{lex}} z\}$ are left for the inputs of g after $g(x)$ is evaluated. If evaluating these $g(z)$ does not require the knowledge of $\{(y, f(y)) \mid y \in S_x\}$, the set $\{(y, f(y)) \mid y \in S_x\}$ become useless and thus can be safely removed from TAB after $f(x)$ is inserted.

We prove Equation 14 case by case:

Case 1: When $z = (z_n, \dots, z_{j+1}, 1, \underbrace{0, \dots, 0}_{j-1}) \in \{z \mid z \in S, x <_{\text{lex}} z\}$ for $i \leq j \leq n$,

according to the evaluation rule $\mathcal{R}_{\text{lex}}$, we have

$$z' = (z_n, \dots, z_{j+1}, 0, \underbrace{0, \dots, 0}_{j-1}) \xrightarrow{\mathcal{R}_{\text{lex}}} z.$$

For $i \leq j$, we have $(z'_i, \dots, z'_1) = (\underbrace{0, \dots, 0}_i) \neq (\underbrace{1, 0, \dots, 0}_{i-1})$, thus resulting in

$z' \notin S_x$.

Case 2: When $z = (z_n, \dots, z_{j+1}, 1, \underbrace{0, \dots, 0}_{j-1}) \in \{z \mid z \in S, x <_{\text{lex}} z\}$ for $1 \leq j < i$,

according to the evaluation rule $\mathcal{R}_{\text{lex}}$, we have

$$z' = (z_n, \dots, z_{j+1}, 0, \underbrace{0, \dots, 0}_{j-1}) \xrightarrow{\mathcal{R}_{\text{lex}}} z.$$

If $(z'_i, \dots, z'_1) \neq (1, \underbrace{0, \dots, 0}_{i-1})$, we directly have $z' \notin S_x$.

If $(z'_i, \dots, z'_1) = (1, \underbrace{0, \dots, 0}_{i-1})$, since $j < i$, we have

$$z' = (z_n, \dots, z_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}), \quad z = (z_n, \dots, z_{i+1}, 1, \underbrace{0, \dots, 0}_{i-j-1}, \underbrace{1, 0, \dots, 0}_{j-1}).$$

Since $x = (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}) <_{\text{lex}} z$, we have $x \leq_{\text{lex}} z'$. Therefore, we also have $z' \notin S_x$. □

Based on Lemma 4, after computing $g(x)$ for $x \in S$ and $x \neq S[0]$, we can remove all elements $(y, g(y))$ satisfying $y \in S_x$ from TAB . This gives us an efficient way to construct and update TAB when evaluating g over $S = P_{n'_q, \dots, n'_2, n'_1}^{w'_q, \dots, w'_2, 0}$.

Construction of $A_{k_0, \dots, k_i}$. From now on, we use $A_{k_0, \dots, k_i}$ to represent the table $TAB_{k_0, \dots, k_i}$ to keep consistency with our algorithm. The table $A_{k_0, \dots, k_i}$ is used to store the results of evaluating $D_{k_0, \dots, k_i}(f)$ over $S_{k_0, \dots, k_i} \in P_{n'_q, \dots, n'_2, k_i}^{w'_q, \dots, w'_2, 0}$. If $i < d$, $A_{k_0, \dots, k_i}$ is an array of size $n + 1 - k_i$, each entry of which has the following property:

- $A_{k_0, \dots, k_i}[0]$ stores $D_{k_0, \dots, k_i}(f)(\mathbf{0}^n) = \mu(\sum_{j=1}^i e_{k_j})$.
- $A_{k_0, \dots, k_i}[j]$ ($1 \leq j \leq n - k_i$) is always updated to store $D_{k_0, \dots, k_i}(f)(x)$ where

$$x = (x_n, \dots, x_{k_i+j+1}, 1, \underbrace{0, \dots, 0}_{j-1}, \underbrace{0, \dots, 0}_{k_i}) \in P_{n'_q, \dots, n'_2, k_i}^{w'_q, \dots, w'_2, 0}$$

at the point when computing $D_{k_0, \dots, k_i}(f)(x)$ is done. The update is equivalent to removing results $(y, D_{k_0, \dots, k_i}(f)(y))$ where

$$y = (y_n, \dots, y_{k_i+j+1}, 1, \underbrace{0, \dots, 0}_{j-1}, \underbrace{0, \dots, 0}_{k_i}), \text{ and } y <_{\text{lex}} x,$$

since such results are no longer useful for subsequent computations.

For each array $A_{k_0, \dots, k_d}$, its size is simply 1, and $A_{k_0, \dots, k_d}[0] = \mu(\sum_{j=1}^d e_{k_j})$.

Update of the tables. Evaluating $f(x)$ is based on Equation 6. Let $x = \sum_{j=1}^h e_{k_j}$ be the current input to f where $1 \leq k_1 < \dots < k_h \leq n$. From Equation 6, we know that the following relation is used for computing $D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)})$:

$$D_{k_0, \dots, k_i}(f)(x^{(k_0, \dots, k_i)}) = D_{k_0, \dots, k_i}(f)(x_{(k_{i+1})}^{(k_0, \dots, k_i)}) \oplus D_{k_0, \dots, k_{i+1}}(f)(x^{(k_0, \dots, k_{i+1})}).$$

Moreover, when reaching this step, both

$$D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}), D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})})$$

have been known (i.e., computed before this step). As they will be used here, they have not yet been removed from the table, and are stored somewhere.

Since $\mathbf{x} = \sum_{j=1}^h \mathbf{e}_{k_j}$, we have

$$\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)} = \sum_{j=i+1}^h \mathbf{e}_{k_j}, \mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)} = \sum_{j=i+2}^h \mathbf{e}_{k_j}, \mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})} = \sum_{j=i+2}^h \mathbf{e}_{k_j}.$$

Therefore, the result $D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)})$ is used to update $A_{k_0, \dots, k_i}[k_{i+1} - k_i]$, i.e.,

$$A_{k_0, \dots, k_i}[k_{i+1} - k_i] = D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) \oplus D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})}). \quad (15)$$

We are left to identify where $D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)})$ and $D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})})$ are stored in order to update $A_{k_0, \dots, k_i}[k_{i+1} - k_i]$ based on [Equation 15](#). Depending on the values of i and $\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}$, there are in total 3 different cases, which explain why 3 different cases are handled in [Algorithm 1](#).

The first case is $\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)} = \mathbf{0}^n$. Then we have

$$D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) = A_{k_0, \dots, k_i}[0], D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})}) = A_{k_0, \dots, k_{i+1}}[0].$$

The second case is $i = d - 1$ and $\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)} \neq \mathbf{0}^n$. Then, we have

$$D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) = A_{k_0, \dots, k_i}[k_{i+2} - k_i], \\ D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})}) = A_{k_0, \dots, k_{i+1}}[0].$$

Otherwise, based on the construction of the tables, we have

$$D_{k_0, \dots, k_i}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_i)}) = A_{k_0, \dots, k_i}[k_{i+2} - k_i], \\ D_{k_0, \dots, k_{i+1}}(f)(\mathbf{x}_{(k_{i+1})}^{(k_0, \dots, k_{i+1})}) = A_{k_0, \dots, k_{i+1}}[k_{i+2} - k_{i+1}].$$

Combining all the above analysis described in [Section 5](#), we get our algorithm described in [Section 3](#) and also prove its correctness.

6 Conclusion

We propose an efficient algorithm to evaluate a Boolean polynomial f in n variables and of degree d on all inputs in a general structured space $P_{n_s, \dots, n_1}^{w_s, \dots, w_1} \subseteq \mathbb{F}_2^n$. The algorithm beats all polynomial evaluation algorithms in time complexity even if $P_{n_s, \dots, n_1}^{w_s, \dots, w_1} = \mathbb{F}_2^n$, and solves an open question raised in [\[2\]](#). In particular, our algorithm can replace an unproven method to solve a critical step in Dinur's polynomial method [\[4\]](#) without affecting its overall time complexity.

Acknowledgement. We thank Itai Dinur for valuable discussions on polynomial evaluations over $P_{n-n_1, n_1}^{w_2, n_1}$, which is related to solving a critical step in his polynomial method.

References

1. Banik, S., Caforio, A., Vaudenay, S.: New attacks on lowmc using partial sets in the single-data setting. *IACR Commun. Cryptol.* **1**(1), 22 (2024)
2. Bouillaguet, C.: Algorithm 1052: Evaluating a boolean polynomial on all possible inputs. *ACM Trans. Math. Softw.* **50**(4), 28:1–28:37 (2024)
3. Bouillaguet, C., Chen, H.C., Cheng, C.M., Chou, T., Niederhagen, R., Shamir, A., Yang, B.Y.: Fast exhaustive search for polynomial systems in $\mathbb{F}_2$. In: Mangard, S., Standaert, F.X. (eds.) *Cryptographic Hardware and Embedded Systems, CHES 2010*. pp. 203–218. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
4. Dinur, I.: Cryptanalytic applications of the polynomial method for solving multivariate equation systems over $\text{GF}(2)$. In: *EUROCRYPT (1)*. *Lecture Notes in Computer Science*, vol. 12696, pp. 374–403. Springer (2021)
5. Dinur, I., Shamir, A.: Cube Attacks on Tweakable Black Box Polynomials. In: *EUROCRYPT*. *Lecture Notes in Computer Science*, vol. 5479, pp. 278–299. Springer (2009)
6. Dinur, I., Shamir, A.: An Improved Algebraic Attack on Hamsi-256. In: *FSE*. *Lecture Notes in Computer Science*, vol. 6733, pp. 88–106. Springer (2011)
7. Furue, H., Takagi, T.: Fast enumeration algorithm for multivariate polynomials over general finite fields. In: Johansson, T., Smith-Tone, D. (eds.) *Post-Quantum Cryptography*. pp. 357–378. Springer Nature Switzerland, Cham (2023)
8. Liu, F., Mahzoun, M., Øygarden, M., Meier, W.: Algebraic Attacks on RAIN and AIM Using Equivalent Representations. *IACR Trans. Symmetric Cryptol.* **2023**(4), 166–186 (2023)
9. Liu, F., Meier, W., Sarkar, S., Isobe, T.: New Low-Memory Algebraic Attacks on LowMC in the Picnic Setting. *IACR Trans. Symmetric Cryptol.* **2022**(3), 102–122 (2022)
10. Lokshtanov, D., Paturi, R., Tamaki, S., Williams, R.R., Yu, H.: Beating Brute Force for Systems of Polynomial Equations over Finite Fields. In: *SODA*. pp. 2190–2202. SIAM (2017)
11. Savage, C.D., Winkler, P.: Monotone Gray Codes and the Middle Levels Problem. *J. Comb. Theory A* **70**(2), 230–248 (1995)

supplementary materials

A Examples for 4-bit Gray code and Monotone Gray Code

The Gray code for 4-bit binary vectors is shown as follows:

$$G_4 = \{ (0,0,0,0), (0,0,0,1), (0,0,1,1), (0,0,1,0), \\ (0,1,1,0), (0,1,1,1), (0,1,0,1), (0,1,0,0), \\ (1,1,0,0), (1,1,0,1), (1,1,1,1), (1,1,1,0), \\ (1,0,1,0), (1,0,1,1), (1,0,0,1), (1,0,0,0) \}.$$

One example of the monotone Gray code for 4-bit binary vectors is shown as follows:

$$MG_4 = \{ \underbrace{(0,0,0,0), (0,0,0,1)}_{V_{0,1}}, \\ \underbrace{(0,0,1,1), (0,0,1,0), (0,1,1,0), (0,1,0,0), (1,1,0,0), (1,0,0,0)}_{V_{1,2}}, \\ \underbrace{(1,0,1,0), (1,0,1,1), (1,0,0,1), (1,1,0,1), (0,1,0,1), (0,1,1,1)}_{V_{2,3}}, \\ \underbrace{(1,1,1,1), (1,1,1,0)}_{V_{3,4}} \}.$$

B Example for Our Algorithm

Let $n = 3$ and

$$f(x_3, x_2, x_1) = x_3x_2 \oplus x_2x_1 \oplus x_2 \oplus x_1.$$

Since $d = 2$, we need to prepare $\binom{3}{\leq 2} = 7$ different tables:

$$A_0, A_{0,1}, A_{0,2}, A_{0,3}, A_{0,1,2}, A_{0,1,3}, A_{0,2,3},$$

and initialize them as follows:

$$A_0[0] = 0, A_{0,1}[0] = 1, A_{0,2}[0] = 1, A_{0,3}[0] = 0, \\ A_{0,1,2}[0] = 1, A_{0,1,3}[0] = 0, A_{0,2,3}[0] = 1.$$

We consider evaluating this f on all inputs in $\mathbb{F}_2^3$ and the corresponding evaluation phase is shown in [Table 2](#).

Table 2: Example of our algorithm for polynomial evaluations

(x_3, x_2, x_1)	$(k_h, \dots, k_1)$	Update	$f(x_3, x_2, x_1)$
(0,0,0)	—	—	$A_0[0] = 0$
(0,0,1)	(1)	$A_0[1] = A_0[0] \oplus A_{0,1}[0] = 0 \oplus 1 = 1$	$A_0[1] = 1$
(0,1,0)	(2)	$A_0[2] = A_0[0] \oplus A_{0,2}[0] = 0 \oplus 1 = 1$	$A_0[2] = 1$
(0,1,1)	(2,1)	$A_{0,1}[1] = A_{0,1}[0] \oplus A_{0,1,2}[0] = 1 \oplus 1 = 0$ $A_0[1] = A_0[2] \oplus A_{0,1}[1] = 1 \oplus 0 = 1$	$A_0[1] = 1$
(1,0,0)	(3)	$A_0[3] = A_0[0] \oplus A_{0,3}[0] = 0 \oplus 0 = 0$	$A_0[3] = 0$
(1,0,1)	(3,1)	$A_{0,1}[2] = A_{0,1}[0] \oplus A_{0,1,3}[0] = 1 \oplus 0 = 1$ $A_0[1] = A_0[3] \oplus A_{0,1}[2] = 0 \oplus 1 = 1$	$A_0[1] = 1$
(1,1,0)	(3,2)	$A_{0,2}[1] = A_{0,2}[0] \oplus A_{0,2,3}[0] = 1 \oplus 1 = 0$ $A_0[2] = A_0[3] \oplus A_{0,2}[1] = 0 \oplus 0 = 0$	$A_0[2] = 0$
(1,1,1)	(3,2,1)	$A_{0,1}[1] = A_{0,1}[2] \oplus A_{0,1,2}[0] = 1 \oplus 1 = 0$ $A_0[1] = A_0[2] \oplus A_{0,1}[1] = 0 \oplus 0 = 0$	$A_0[1] = 0$

C Examples for Understanding Algorithm 3

Example 1. We first take the method in [3] as an instance, and consider evaluating a quadratic polynomial $f(x_3, x_2, x_1)$ in the full space $\mathbb{F}_2^3$, i.e., $d = 2$ and $n = 3$. Then, the ordered set S_0 (i.e., Gray code G_3 for $\mathbb{F}_2^3$) is

$$S_0 = \{(0,0,0), (0,0,1), (0,1,1), (0,1,0), (1,1,0), (1,1,1), (1,0,1), (1,0,0)\}.$$

It can be observed that any two consecutive elements in S_0 differs in only 1 bit. The evaluation rule $\mathcal{R}$ over a set S is defined as:

$$\forall j \in [1, |S| - 1] : S[j - 1] \xrightarrow{\mathcal{R}} S[j].$$

With Algorithm 3, by setting $S_{0,i} = \text{NEXTSET}(S_0, \mathcal{R}, i)$, we can get

$$\begin{aligned} S_{0,1} &= \{(0,0,0), (0,1,0), (1,1,0), (1,0,0)\}, \\ S_{0,2} &= \{(0,0,1), (1,0,1)\}, \\ S_{0,3} &= \{(0,1,0)\}, \end{aligned}$$

which are the input sets to polynomials $D_{0,1}(f), D_{0,2}(f), D_{0,3}(f)$, respectively.

It can be observed that any two consecutive elements in $S_{0,1}, \dots, S_{0,3}$ still differ in 1 bit. Therefore, the same evaluation as $\mathcal{R}$ rule can be reused for the sets $S_{0,1}, S_{0,2}, S_{0,3}$. With Algorithm 3 for these 3 sets, we can similarly get

$$S_{0,1,2} = \{(0,0,0), (1,0,0)\}, S_{0,1,3} = \{(0,1,0)\}, S_{0,2,3} = \{(0,0,1)\}$$

which are the input sets for polynomials $D_{0,1,2}(f), D_{0,1,3}(f), D_{0,2,3}(f)$, respectively. Note that $D_{0,1,2}(f), D_{0,1,3}(f), D_{0,2,3}(f)$ are constants since the degree of f is 2.

For the initialization phase, the following 4 outputs have to be computed and stored:

$$f(0,0,0), D_{0,1}(f)(0,0,0), D_{0,2}(f)(0,0,1), D_{0,3}(f)(0,1,0).$$

Example 2. We then consider the method in [7] and we still use $n = 3$ and $d = 2$. Then, the ordered set S_0 is

$$S_0 = \{(0,0,0), (0,0,1), (0,1,0), (0,1,1), (1,0,0), (1,0,1), (1,1,0), (1,1,1)\}.$$

The evaluation rule $\mathcal{R}$ over S is defined as

$$(x_n, \dots, x_{i+1}, 0, \underbrace{0, \dots, 0}_{i-1}) \xrightarrow{\mathcal{R}} (x_n, \dots, x_{i+1}, 1, \underbrace{0, \dots, 0}_{i-1}).$$

As a result, by setting $S_{0,i} = \text{NEXTSET}(S_0, \mathcal{R}, i)$, we have

$$\begin{aligned} S_{0,1} &= \{(0,0,0), (0,1,0), (1,0,0), (1,1,0)\}, \\ S_{0,2} &= \{(0,0,0), (1,0,0)\}, \\ S_{0,3} &= \{(0,0,0)\}. \end{aligned}$$

As the degree of f is 2, we only need to consider the non-constant polynomials $D_{0,1}(f), D_{0,2}(f), D_{0,3}(f)$, whose input sets are $S_{0,1}, S_{0,2}, S_{0,3}$, respectively.

At the initialization phase, it is required to compute

$$f(0,0,0), D_{0,1}(f)(0,0,0), D_{0,2}(f)(0,0,0), D_{0,3}(f)(0,0,0).$$